

Projeto de Sistemas de Software

Jânio Coutinho Canuto

2025

Sumário

1	Introdução	1
1.1	Abordagem	1
1.2	O programa e o <i>Software</i>	2
1.2.1	A crise do <i>software</i>	3
1.3	Engenharia de Software	4
1.4	Produtos de <i>Software</i>	4
1.4.1	Produtos de qualidade	5
1.4.2	Processo de produção	6
1.5	Ética Profissional	7
1.6	Recapitulando	8
2	O Processo de <i>Software</i>	9
2.1	Atividades do Processo	9
2.1.1	Especificação	10
2.1.2	Projeto	10
2.1.3	Implementação	11
2.1.4	Validação	12
2.1.5	Evolução	12
2.2	Tipos de Processo	12
2.3	Modelos de Processo	13
2.3.1	Desenvolvimento em cascata	13
2.3.2	Desenvolvimento incremental	15
2.3.3	Desenvolvimento orientado a reuso	16
2.4	Métodos Ágeis	16
2.4.1	Extreme Programming (XP)	17
2.5	Gerência de Projetos Ágeis	19
2.5.1	Scrum	19
2.6	Qual Metodologia Utilizar?	20
2.6.1	Fatores de Sistema	20
2.6.2	Fatores Humanos	20
2.6.3	Fatores Organizacionais	21

2.6.4	Mitos do Desenvolvimento	21
2.6.5	Princípios de Hooker	21
2.7	Recapitulando	22
3	Engenharia de Requisitos	25
3.1	Tipos de Requisitos	25
3.1.1	Requisitos funcionais	25
3.1.2	Requisitos não-funcionais	26
3.2	O Processo de Requisitos	27
3.2.1	Elicitação e Análise	27
3.2.2	Especificação	28
3.2.3	Validação	30
3.3	Gerenciamento de Requisitos	31
3.4	Recapitulando	31
4	Projeto de <i>Software</i>	33
4.1	Orientação a Objetos (OO)	33
4.2	Especificação x Projeto	35
4.3	Alguns Tipos de Modelo	35
4.3.1	Modelos de Contexto	36
4.3.2	Modelos de Processo	36
4.3.3	Modelos Comportamentais	36
4.3.4	Modelo de Dados	39
4.3.5	Modelos de Objetos	39
5	CRUD	45
5.1	Um CRUD simples	46
5.2	Persistência de Dados	53
5.2.1	Arquivos	53
5.2.2	Escrevendo Arquivos no JavaScript	54
5.2.3	JSON	57
6	Bancos de Dados	61
6.1	Tipos	62
6.2	Propriedades ACID	63
6.3	O Modelo Relacional	64
6.3.1	Conceitos Fundamentais	64
6.3.2	Propriedades	65
6.3.3	Modelagem de Entidades e Relacionamentos	65
6.4	Conclusão	71

SUMÁRIO

v

A SQL

73

Capítulo 1

Introdução

Nesta disciplina vamos apresentar alguns conceitos fundamentais no desenvolvimento profissional de *softwares*, uma grande área normalmente chamada de Engenharia de Software.

Dentro da Engenharia de Software, veremos de algumas partes fundamentais do processo de produção como o levantamento de requisitos, o projeto da interface com o usuário, o projeto de bancos de dados e os princípios éticos desta profissão.

1.1 Abordagem

Há inúmeros livros e artigos disponíveis acerca deste tema, inclusive cursos profissionalizantes, superiores e especializações inteiras sobre cada uma das sub-áreas que vamos ver aqui.

Ao invés de simplesmente replicar o que há na literatura acadêmica, proponho que foquemos nas ideias fundamentais e não tanto nos termos utilizados. Isso não quer dizer que não vou apresentar os termos, vou sim, mas o mínimo que deve ficar é a ideia geral por trás de cada um desses termos.

Vocês perceberão que muitas das ideias são bastante intuitivas e dependem de bom-senso. Além disso, também perceberão que boa parte das ideias que veremos podem ser aplicadas em outros contextos, além do desenvolvimento de *software*.

Vamos tentar desenvolver quatro habilidades paralelas à programação mas que são fundamentais para qualquer bom projeto:

- Comunicação,
- Planejamento, e
- Gerenciamento.

Portanto, essa disciplina será um pouco menos computacional e mais próxima da administração. Mas na verdade todo bom programador tem e usa essas habilidades no seu dia-a-dia.

A *comunicação* é provavelmente a ferramenta mais incrível já desenvolvida é através dela que transmitimos ideias de um indivíduo a outro e é também através dela que transmitimos ideias ao nosso computador. Quando escrevemos um código, é fundamental que sejamos capaz de descrever de forma clara e sem erros o que a máquina deve fazer.

O *planejamento* é necessário para que a ordem na qual fazemos as coisas nos leve até o resultado esperado. Um programa nada mais é que um planejamento de ações tomadas pela máquina. Além disso, um bom planejamento evita surpresas e re-trabalhos.

O *gerenciamento* é necessário para garantir que aquilo que foi planejado está sendo executado adequadamente. Em geral, um bom gerenciamento é capaz de melhorar a produtividade e reduzir custos.

Por enquanto é isso, vamos iniciar nossa história entendendo primeiro o que é o tal do *software*. Você sabe?

1.2 O programa e o *Software*

Quando aprendemos uma linguagem de programação, conhecemos técnicas que nos permitem, com um pouco (ou muito) de trabalho, identificar um problema e criar um **algoritmo** para ser **implementado** nesta linguagem e obter um **programa** que o resolve.

Mas essas palavras destacadas não são sinônimos? Sim... e não. No contexto em que apareceram *algoritmo* é uma descrição de uma sequência de passos que resolve o problema, *implementação* é a codificação do algoritmo numa linguagem específica e *programa* é o que de fato executamos para resolver o problema. Mas atenção, em muitas situações elas podem, e são, utilizadas como sinônimos sim.

E o *software*, não é a mesma coisa em inglês? Mais uma vez, sim e não. Em muitas situações isso é verdade, mas no contexto do desenvolvimento profissional *software* é um produto que consiste no programa e em alguns outros elementos associados à produção dele (normalmente chamados de documentação).

A diferença é sutil, mas esse entendimento de que *software* é um produto e não apenas um passa-tempo ou projeto individual levou um tempo para ocorrer e mudou o mercado.

1.2.1 A crise do *software*

No início da computação, os programas eram desenvolvidos, em sua maioria, de forma individual ou dentro de uma empresa para seus próprios fins. Muitas vezes isso era feito sem muito cuidado com a organização do processo de desenvolvimento. Enquanto os programas eram simples, tudo ia bem, apesar de que eventualmente ocorria de alguém sair da empresa e ninguém mais saber como alterar o programa.

Conforme o tempo vai passando, as demandas de funcionalidades dos programas vão aumentando, o que vai tornando os sistemas cada vez maiores e mais complexos.

Quanto maior o gigante, maior a queda. Poucas pessoas trabalhando em sistemas gigantes começaram a gerar muito atraso. Fácil, basta contratar mais gente. Mas, muita gente trabalhando de forma desorganizada só gera mais atraso, e também custos maiores. Além disso, a quantidade de falhas só tende a aumentar.

Imagine a seguinte situação, a pessoa A faz um programa pra receber os dados do usuário num certo formato. A pessoa B faz um programa para calcular a média de um conjunto valores fornecidos em um formato diferente daquele que a pessoa A usou. Quando conecta as duas partes, o resultado final é um erro.

Falhas em produtos físicos podem ser resolvidas com troca de peças, elas surgem normalmente com o desgaste do sistema. Num produto intangível, ou seja, que não pode ser tocado, como um programa de computador não surgem com o tempo pois não há desgaste. Se uma falha surge é porque ela sempre esteve lá e a única solução possível é refazer o produto, ou pelo menos parte dele.

A depender do tipo de sistema, falhas de *software* podem ser catastróficas. Imagine um controle de foguete ou equipamento médico, ou um programa para cálculo estrutural, sem contar todo o sistema financeiro.

O cenário então é o seguinte, temos uma economia global cada vez mais dependente de *softwares*, demandando sistemas cada vez mais complexos, com menos tempo disponível e com necessidades que mudam frequentemente.

Esse cenário, em conjunto com o modo de produção desordenado, acarretou em programas com muitas falhas que eram entregues fora do prazo, com um custo acima do previsto e de difícil manutenção. Ou seja, profissionais com baixa produtividade entregando produtos de baixa qualidade. Essa é a crise do *software*.

Em uma reunião da OTAN, em 1968, propõe-se uma solução para esta crise, uma nova disciplina chamada de Engenharia de Software.

1.3 Engenharia de Software

Apesar de nova em seu nome e aplicação, as ideias são bastante antigas. Ao adotar uma visão de mercado, de que o *software*, apesar de diverso e intangível, nada mais é que um produto feito por seres humanos.

Qual a área do conhecimento que trata do processo de confecção de produtos? Pois bem, é a Engenharia. Normalmente associamos o termo engenharia a matemática e cálculos, mas na realidade é a disciplina que trata da elaboração e aplicação de processos para a produção de produtos de qualidade definida segundo alguma demanda.

Uma característica destes processos de produção é que diversos artefatos são gerados ao longo do caminho, como demandas, plantas, projetos, fórmulas, testes, problemas, soluções, alterações de demanda, alterações de projeto, guia de utilização, etc. Estes artefatos são uma *documentação* do processo de produção.

Cada Engenharia (civil, elétrica, química, mecânica, etc.) possui sua área de atuação, mas no fim, essa área determina o tipo de produto que é realizado. Daí surge o termo Engenharia de *Software* (ES).

A ES se preocupa com todos os aspectos da produção de um programa com o objetivo de obter produtos de qualidade com custos e prazos previsíveis. Apesar do foco ser o *software*, as ideias fundamentais são as mesmas de qualquer processo de produção.

Por fim, podemos dizer que o *software*, enquanto produto, é um programa de computador, assim como toda documentação associada à sua produção.

1.4 Produtos de *Software*

Um produto, seja de *software* ou não, surge a partir de duas vias: demanda ou oportunidade.

Um produto sob demanda, ou personalizado, é solicitado por um cliente específico para satisfazer suas necessidades. Quem define os requisitos é o cliente.

Um produto de oportunidade, ou genérico, é produzido para atender, ou criar, uma demanda do mercado. Este produto é então divulgado e vendido para quem quiser usar. Os requisitos são definidos pela empresa que decide produzir, o processo de produção normalmente precisa ser rápido para não perder a oportunidade identificada.

Dissemos que a Engenharia se preocupa em entregar produtos de qualidade. Mas o que significa isso?

1.4.1 Produtos de qualidade

A qualidade de um produto em geral é algo subjetivo, no entanto, há algumas características que são fundamentais para que a percepção de qualidade seja boa:

- Segurança,
- Eficiência,
- Manutenibilidade,
- Aceitabilidade,

O mínimo que se espera de um produto é que ele faça sem erros aquilo para o qual ele foi projetado. Isso nem conta como critério de qualidade, afinal de contas, se um produto não faz o que deve fazer ele simplesmente é inservível. Vamos discutir um pouco sobre os outros critérios e entender como isso se reflete nos programas computacionais.

A segurança de um produto diz respeito à sua capacidade de garantir a integridade (física e moral) do usuário e também evitar que terceiros o usem para fins maliciosos. O uso do produto, mesmo em caso de falha, não deve causar danos. Em sistemas computacionais, além do aspecto físico, diz respeito à proteção dos dados dos usuários e do acesso indevido a sistemas.

A eficiência de um produto diz respeito ao consumo de insumos necessários para cumprir sua finalidade. Pode ser energia, combustível, matéria prima, tempo, etc. Em computação a eficiência diz respeito ao consumo de recursos como processador, memória e largura de banda. Às vezes o termo eficiência também se refere a quão rápido o usuário consegue realizar a tarefa usando o produto. No caso de programas, quantos cliques, ou comandos, são necessários para ativar a ação desejada.

A manutenibilidade é a facilidade de manutenção ou alteração do produto. Em *software* essa característica é muito importante pois é raro que as necessidades do cliente sejam estáticas.

Por fim, a aceitabilidade é o quão bem o usuário aceita o produto. Diversos fatores podem influenciar isto, inclusive os anteriores, mas quase sempre o grande responsável pela aceitabilidade é o *design*. Incluindo a forma de interação, a aparência se é fácil de entender e se é compatível com outros produtos utilizados pelo usuário.

Outro ponto da afirmação sobre a Engenharia é que ela aplica um processo para obter o produto. Que processo é esse?

1.4.2 Processo de produção

O processo de concepção e confecção de um produto, invariavelmente, passa por quatro etapas:

- Especificação: Definir “o que” deve ser feito e com quais restrições
- Desenvolvimento: Projeto e Implementação
 - Projeto: Definir “como” fazer o que deve ser feito
 - Implementação: Fazer o que foi planejado
- Validação: Verificar se o que foi implementado está correto
- Evolução: Modificar para atender novas demandas

A questão agora é, como exatamente cada uma destas etapas acontece? Quanto tempo gastamos em cada uma? Quão detalhado deve ser o projeto? Que tipo de testes fazer? E a resposta pra tudo isso é... depende.

Depende de que? Do tipo de produto, das limitações impostas, do cliente, da equipe envolvida no desenvolvimento. Não existe um processo único, melhor que todos os outros.

Mas o tipo de produto nós já sabemos que é *software*! Não é tão simples assim... *Software* é um dos produtos mais diversos que existe. Pode variar de um pequeno programa embarcado num sensor de temperatura até um sistema de comunicação global.

Temos um arsenal de ferramentas e devemos escolher, e adaptar, as que forem mais adequadas ao cenário enfrentado.

Apesar do processo único não existir, alguns princípios são fundamentais e devem ser considerados em qualquer desenvolvimento:

- Entenda claramente o que é desejado pelo cliente,
- Segurança e eficiência são sempre importantes,
- Reutilize produtos já desenvolvidos, quando adequado,
- O processo escolhido precisa ser bem definido, entendido e gerido pela equipe.

Antes de continuar com as tecnicidades da Engenharia de Software, vamos partir para um assunto fundamental que às vezes é deixado de lado.

1.5 Ética Profissional

O desenvolvimento de *software* é uma profissão e, como em qualquer outra, devemos agir de forma honesta e responsável para ser respeitado como profissional.

Como outras atividades de Engenharia, é mais do que simplesmente aplicar habilidades técnicas para obter um produto. As suas ações afetam as vidas de outras pessoas, direta ou indiretamente.

Se comportar de forma ética não é apenas seguir a lei, isso é agir de forma legal. O comportamento ético envolve seguir um conjunto de princípios que são moralmente corretos.

Há um código de ética do Engenheiros de Software escrito por dois grandes organismos internacionais o IEEE (*Institute of Electrical and Electronics Engineers*) e a ACM (*Association for Computing Machinery*) e também um da SBC (Sociedade Brasileira de Computação).

Em resumo, eles elencam alguns princípios que devem ser observados pelos profissionais da área:

1. Agir consistentemente com o interesse público.
2. Agir no melhor interesse de seu cliente e empregador.
3. Garantir a qualidade de seus produtos.
4. Manter integridade e independência em seu julgamento.
5. Gerentes e líderes devem ser éticos no processo de gerência do desenvolvimento.
6. Promover a integridade e reputação da profissão.
7. Ser justo e apoiar os colegas.
8. Aprender constantemente.

É claro que estes princípios não resolvem todos os problemas e em certas situações eles podem ser conflitantes entre si. Ao trabalhar em um sistema de armas, estou agindo de acordo com o interesse público e de acordo com meus princípios? Se eu discordo das políticas do gerente, devo fazer do meu jeito ou seguir o que ele manda? Se o empregador libera um sistema sem testar, devo avisar ao cliente? Se o sistema é crítico, devo avisar ao público?

Nenhuma destas perguntas têm resposta correta, cabe ao julgamento de cada um decidir o que fazer. Muitos dilemas podem ser resolvidos através de uma comunicação eficiente entre os envolvidos. Agir de forma ética é difícil e às vezes custoso.

1.6 Recapitulando

- *Software* é um produto.
- Queremos produzir um produto de qualidade, que atende uma demanda, e tem custo e prazo bem definidos.
- um produto de qualidade deve ter segurança, eficácia, aceitabilidade e manutenibilidade.
- Aplicamos um processo de produção que quase sempre tem as etapas de
 - Especificação,
 - Projeto,
 - Implementação,
 - Validação, e
 - Evolução.
- O processo não é único para todos os *softwares*, mas sempre devemos
 - entender bem a demanda
 - definir claramente o processo adotado
 - garantir que a equipe entendeu tanto a demanda quanto o processo escolhido
 - garantir que o processo é bem gerido.

Capítulo 2

O Processo de *Software*

Um processo é um conjunto de atividades realizadas para a produção de algum resultado. O processo de *software*, portanto, são as atividades realizadas no desenvolvimento de um produto de *software*.

2.1 Atividades do Processo

Normalmente descrevemos um processo através de suas atividades. Além do nome das atividade, podemos incluir também o resultado da atividade, chamado de produto ou artefato; as responsabilidades das pessoas envolvidas na atividade, chamadas de papéis; e as pré e pós condições, que são verdades antes e depois da realização da atividade.

Existe uma variedade de processos de produção distintos, mas todos eles têm em comum as seguintes atividades:

- Especificação,
- Projeto,
- Implementação,
- Validação,
- Evolução

Como cada atividade gera um produto, elas também podem ser vistas como processos individuais, que possuem suas próprias atividades. É uma questão de ponto de vista.

Bom, mas o que ocorre nestas atividades, quais seus resultados? Vamos ver uma descrição breve delas nas seções seguintes.

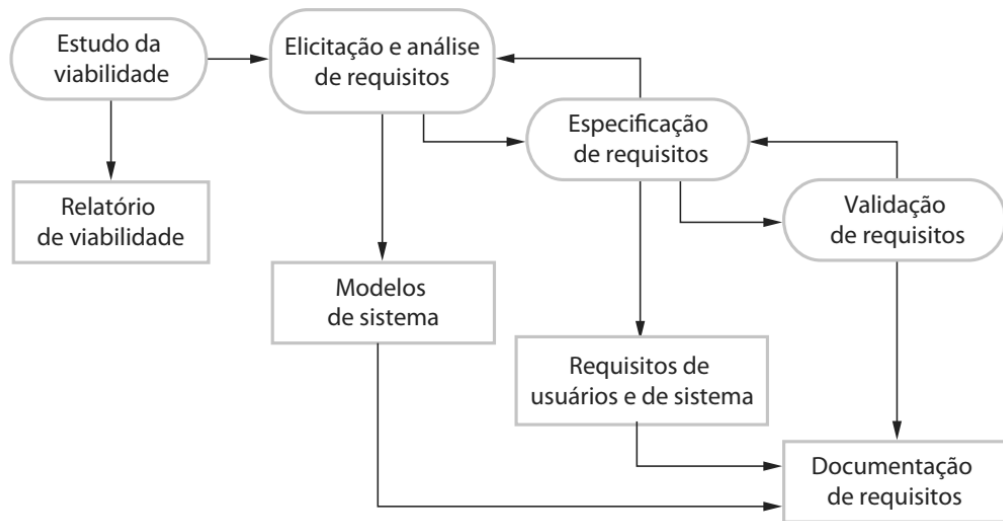


Figura 2.1: Atividades da especificação

2.1.1 Especificação

Na etapa de especificação nós definimos “o que” o sistema deve fazer. Ela costuma iniciar com uma coleta inicial de informações e uma discussão com o cliente acerca de suas necessidades (Elicitação e Análise de requisitos), com isso obtemos um modelo do sistema. Paralelamente a isso, e utilizando estas informações, é feita um estudo de viabilidade, tanto técnica quanto econômica para saber se é possível construir o sistema com o orçamento, tempo e tecnologia disponíveis, gerando um relatório de viabilidade.

Passada esta etapa, faz-se uma especificação dos requisitos, um detalhamento das funcionalidades e características que o usuário deseja e das restrições do sistema, gerando os requisitos de usuário e de sistema. Por fim é feita uma validação dos requisitos levantados junto ao cliente, para verificar se eles de fato representam aquilo que ele deseja. Estes processos, na realidade, costumam ser intercalados podendo levar algumas iterações.

Estando todos de acordo, o produto final do processo de especificação é chamado de Documento de Requisitos, e traz o modelo do sistema e um detalhamento dos requisitos levantados junto ao cliente. A figura 2.1 ilustra este processo.

2.1.2 Projeto

Esta atividade, ilustrada na figura 2.2, determina “como” o sistema deve ser implementado. As grandes atividades aqui são o projeto de arquitetura, onde

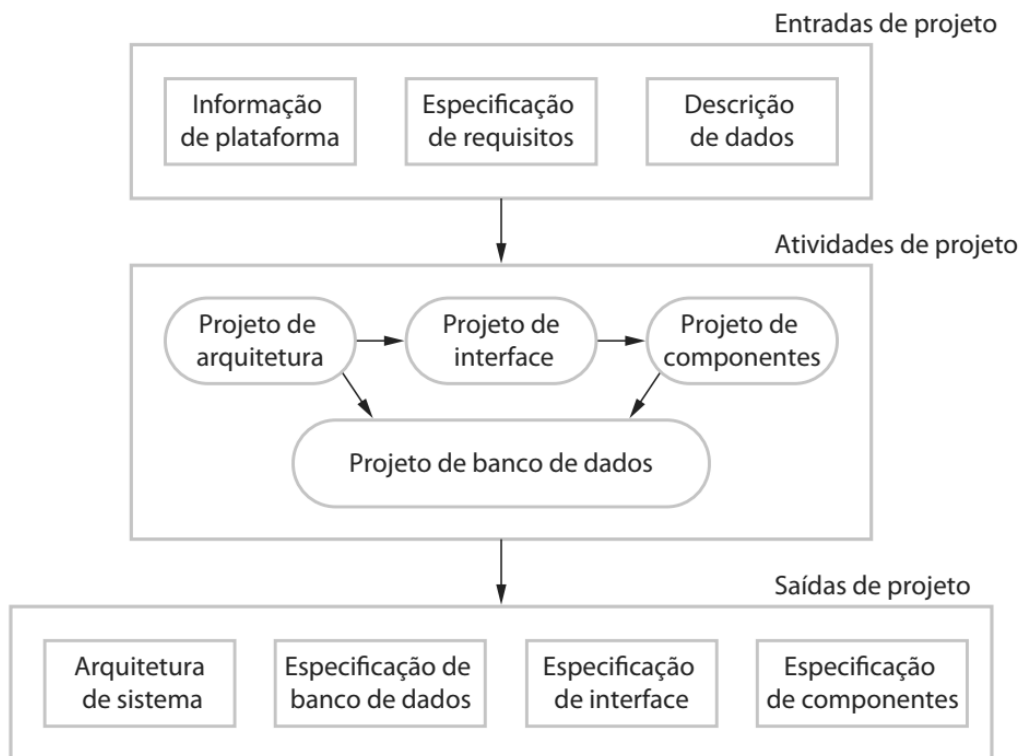


Figura 2.2: Atividades do projeto

temos uma estrutura geral do sistema, quem são os componentes principais e como eles se relacionam. O projeto de interface define como os componentes comunicam-se entre si, pensando em termos de funções, é uma especificação das entradas e saídas. Uma interface bem projetada permite que os componentes sejam implementados simultaneamente.

No projeto de componente temos a definição do funcionamento de cada elemento do *software*, se suficientemente detalhado, o modelo do componente já é quase uma implementação, faltando apenas traduzir para uma linguagem de programação específica. O projeto de banco de dados determina como os dados são representados. Cada uma destas atividades produz um documento, descrevendo os resultados obtidos.

2.1.3 Implementação

Esta atividade é a que torna o produto real. Aqui os componentes projetados são implementados de forma independente e posteriormente integrados no sistema final. Na prática, implementação e projeto costumam ser inter-

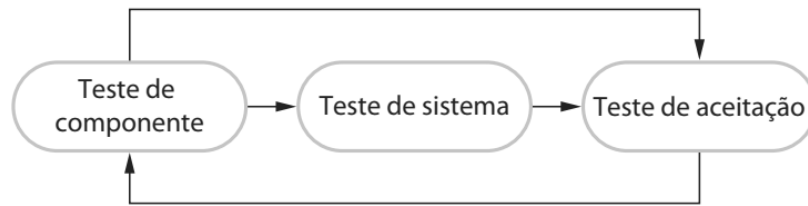


Figura 2.3: Atividades da validação

calados.

2.1.4 Validação

A etapa de validação costuma ser intercalada com o final da implementação. Três tipos de testes costumam ser realizados, o primeiro é o teste dos componentes individuais, para verificar se eles atendem àquilo que foi projetado; o segundo é o teste do sistema como um todo, depois que os componentes são conectados, para verificar se o conjunto continua correto e atende aos requisitos; e por fim, o teste de aceitação envolve a utilização de dados reais providos pelo cliente e da avaliação de usuários.

O processo de avaliação é ilustrado na figura 2.3. Vencida esta etapa, o sistema está pronto para ser entregue.

2.1.5 Evolução

Como já discutimos, mudanças podem ocorrer a qualquer momento, durante ou após a produção do sistema. Historicamente a evolução, ou manutenção, é vista como um processo separado, e desinteressante, do desenvolvimento.

Hoje em dia essa distinção é quase irrelevante, poucos sistemas são totalmente novos e faz mais sentido pensar em desenvolvimento e manutenção como um processo contínuo, um processo evolutivo onde o *software* é alterado frequentemente em resposta a mudanças nos requisitos, como ilustrado na figura

2.2 Tipos de Processo

Às vezes os processos são categorizados em baseados em planos ou ágeis, como na figura 2.5. Nos processo dirigido a planos, todas as atividades são planejadas com antecedência e o progresso é avaliado em comparação com

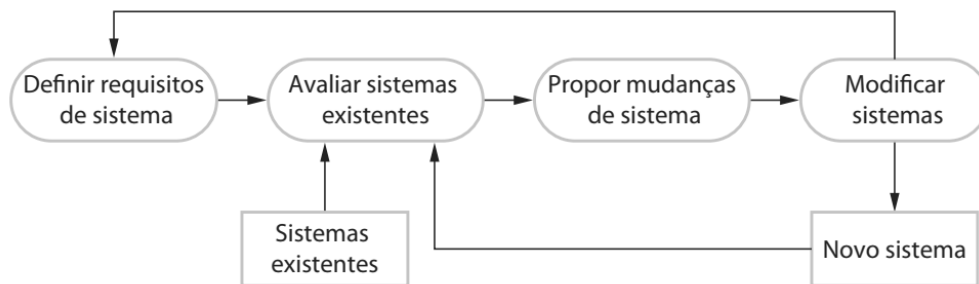


Figura 2.4: Atividades da evolução

o planejamento inicial. Este tipo de abordagem é particularmente útil em sistemas críticos ou muito grandes.

Já nos processos ágeis, o planejamento é gradativo e, por isso, é mais fácil acomodar alterações no processo. Cada abordagem é mais adequada para um tipo de produto, sendo na prática utilizado uma mistura entre as duas abordagens.

2.3 Modelos de Processo

Atividades, por si só, não são o processo. É preciso saber como essas atividades são executadas e como elas comunicam-se. Os modelos de processo são descrições gerais da organização de suas atividades.

Aqui apresento três modelos, em cascata que invariavelmente é baseado em planos, e dois modelos que podem ser baseados em planos ou ágeis, o modelo incremental e o modelo orientado a reuso.

2.3.1 Desenvolvimento em cascata

Neste modo de trabalhar, cada atividade só se inicia após o término da anterior, conforme ilustrado na figura 2.6. Cada estágio produz um documento que deve ser aprovado pela equipe e pelos clientes. Na prática, estes acabando sendo modificados ao longo da produção para refletir alterações realizadas em atividades posteriores.

As etapas de implementação e validação são divididas em duas atividades sequenciais. Primeiro a implementação dos componentes os teste unitários, nos elementos isolados. Depois, a integração dos componentes e o teste do sistema como um todo.

Em geral este tipo de projeto é mais lento, mas dá mais garantia de qualidade sobre o produto final e, se bem feito, é mais fácil de manter e

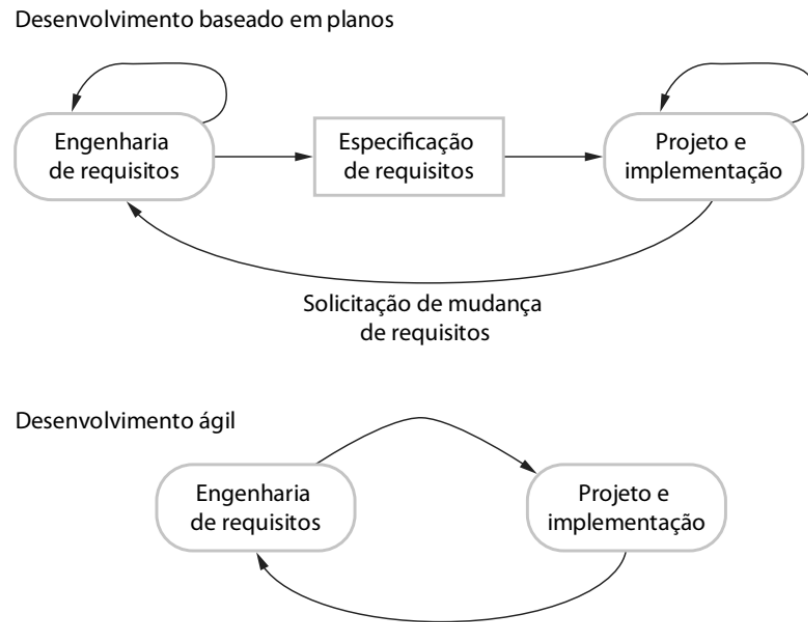


Figura 2.5: Tipos de processo

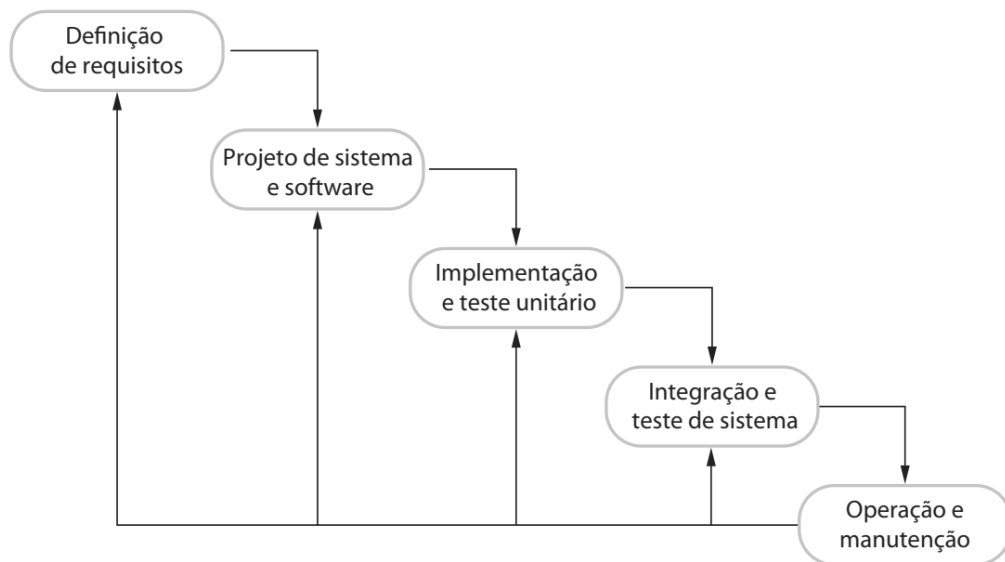


Figura 2.6: Modelo em cascata

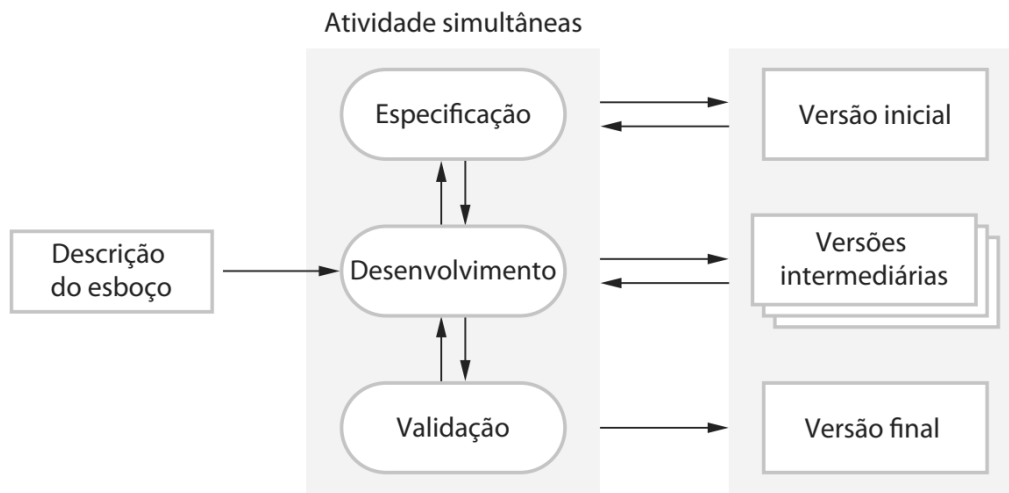


Figura 2.7: Modelo incremental

evoluir.

As vantagens deste modelo é que o processo é visível, com progresso facilmente monitorável e uma extensa documentação. Em contrapartida, é pouco flexível, pode bloquear parte da equipe, que aguarda alguma etapa ser concluída, e exige paciência do cliente, que só verá o produto no fim do processo.

2.3.2 Desenvolvimento incremental

Neste modelo, ilustrado na figura, as etapas de especificação, desenvolvimento e validação são realizadas de forma intercalada. Inicia-se com um esboço do sistema e produz-se uma versão inicial. Esse protótipo é apresentado ao cliente e, com o *feedback* dele, repensamos os requisitos e repetimos o processo, produzindo uma nova versão do produto. Isso se repete até que se obtenha um sistema satisfatório.

Em geral, cada nova versão adiciona algo à versão anterior, daí o nome incremento. As funcionalidades adicionadas normalmente seguem alguma ordem de prioridade, definida pelo cliente.

Essa forma de trabalhar é mais próxima do raciocínio natural para resolver problemas, tentamos nos aproximar cada vez mais da solução e, se percebemos um erro, damos um passo para trás e tentamos uma rota alternativa. Para a maioria das aplicações comerciais e pessoais esse modelo costuma ser mais adequado.

As vantagens dessa abordagem são a facilidade de obter *feedback*, acomodo-

dar mudanças e a velocidade de entrega de um produto útil, às vezes mesmo antes de ser finalizado. Isso aumenta a satisfação do cliente.

As desvantagens são duas. O processo não é visível, pois não é viável documentar tudo se está sempre mudando, isso dificulta o gerenciamento. Outro ponto importante é que a estrutura geral do sistema tende a se degradar com a adição de novas características. Se não for gasto algum tempo na reorganização periódica do código, fica cada vez mais difícil a manutenção.

2.3.3 Desenvolvimento orientado a reuso

Como o nome sugere, este modo foca em integrar *softwares* já prontos para criar um novo sistema. Obviamente, muitas vezes são necessárias adaptações nos elementos reutilizados para que se adaptem aos requisitos do novo sistema. Ultimamente esta tem sido a abordagem padrão em muitas áreas de desenvolvimento.

Os tipos de objetos reutilizáveis são diversos, podem ser desde bibliotecas de funções até sistemas *stand-alone* ou *web services* disponíveis para chamada remota.

O diagrama do modelo orientado a reuso é apresentado na figura 2.8. Perceba que as etapas de especificação e validação são as mesmas dos outros processos, mas os estágios intermediários são um pouco diferentes.

Na análise de componentes, busca-se elementos que implementem alguns dos requisitos. Na modificação de requisitos verificamos o que não pode ser feito com os componentes identificados e negociamos com o cliente uma alteração nos requisitos iniciais.

O projeto é similar aos outros modelos, mas aqui temos possíveis modificações em componentes existentes e criação de componentes novos, se necessário. Agora a integração faz parte do desenvolvimento, pois é esse o desenvolvimento que fazemos com os componentes reutilizados.

As vantagens são a redução de riscos e custos, além de uma entrega mais rápida. Tudo isso é consequência de criarmos menos componentes novos.

A principal desvantagem é que às vezes o sistema pode não atender algum requisito do cliente por limitações dos componentes disponíveis, além de não termos controle sobre a evolução dos elementos reutilizados.

2.4 Métodos Ágeis

Além dos modelos que vimos há alguns outros, mas no fim das contas, os três apresentados resumem bem o que encontramos na prática. O modelo

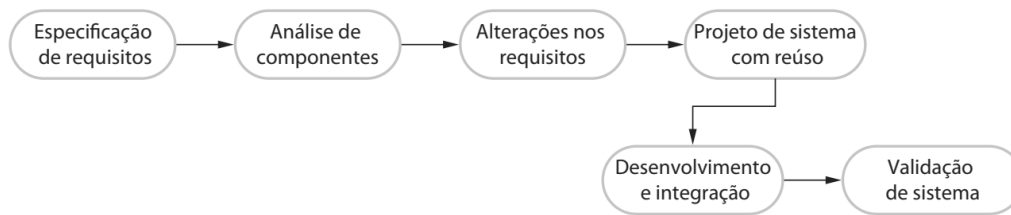


Figura 2.8: Modelo orientado a reuso

incremental está diretamente ligado aos processos ágeis, que discutimos na seção 2.2.

A engenharia de *software* tradicional é baseada em planos, provavelmente uma consequência de que aqueles que a definiram trabalhavam em grandes sistemas. Essa abordagem gera uma grande sobrecarga na elaboração de uma documentação detalhada. Daí, um grupo de desenvolvedores propõe os métodos ágeis que focam no programa, e não no seu projeto e documentação.

Mas isso não é voltar à estaca zero, à situação que deu origem à crise do *software*? Possivelmente sim, mas se feito com cuidado, não.

Os princípios dos métodos ágeis são um grande envolvimento do cliente no projeto, desenvolvimento e entregas incrementais, focar nas pessoas e não nos processos, aceitar que as mudanças vão ocorrer e manter a simplicidade do código gerado.

A ideia é que se você mantém um código limpo, bem escrito e estruturado, ele já é a própria documentação. O foco nas pessoas envolve reconhecer as preferências e habilidades da equipe desenvolvedora, deixar cada um trabalhar do seu jeito e não da forma definida pelo processo.

Vamos conhecer agora um dos métodos ágeis mais conhecidos e utilizados.

2.4.1 Extreme Programming (XP)

A programação extrema (XP) é provavelmente um dos métodos ágeis mais conhecidos e utilizados, ou variações dela. A ideia é levar algumas práticas de desenvolvimento ao extremo, o que na realidade nem sempre é possível. Veremos agora alguma dessas práticas.

Na XP os requisitos são expressos como cenários (estórias do usuário), que por sua vez são quebrados em tarefas. A ideia é que o cliente é parte do time de desenvolvimento e expressa o que quer através de exemplos da interação desejada (estórias). O time de desenvolvimento analisa estes cenários e identifica as tarefas que o *software* deve realizar para que a estória se concretize. O cliente deve ordenar os cenários na forma que é mais importante para ele e escolher qual a próxima estória a ser incluída na entrega seguinte.

Os programadores trabalham em pares e iniciam o criam testes para as tarefas antes de implementá-las. O desenvolvimento orientado a testes, como é chamado, consiste em desenvolver os testes antes das funções e, idealmente, devem contar com a participação do cliente. Estes testes normalmente são automatizados, o que permite a realização de todos os testes a cada nova entrega. Um problema dos testes é que muitas vezes são incompletos, ou seja, não cobrem todos os casos possíveis; e, para certas tarefas, muito difíceis de escrever.

A programação em pares, como o nome diz, trata-se de programadores trabalhando em duplas. Essas duplas são dinâmicas, mudando ao longo do projeto. As vantagens do trabalho em dupla é que serve como um sistema de revisão imediato, já que um observa enquanto o outro escreve e, como tem alguém lendo, encorajam a escrita de código legível e organizado. Além disso, por serem dinâmicas, o conhecimento do sistema como um todo fica compartilhado pela equipe, minimizando os problemas quando alguém precisa sair do time.

Após o desenvolvimento, os componentes (elementos que efetuam as tarefas) são integrados, todos os testes são realizados, e o cliente recebe e avalia o produto. Essas entregas de novas versão ocorrem em intervalos curtos, tipicamente de 15 dias.

Um dos problemas do desenvolvimento incremental é a degradação da estrutura do sistema como um todo, o que atrapalha a manutenção posterior. Na ES tradicional diz-se que devemos nos planejar para mudanças futuras, a XP diz que não vale a pena perder esse tempo porque não dá pra prever de forma confiável. Simplesmente faça as alterações quando elas surgirem. Então como resolver o problema da degradação?

A XP propõe que se faça um melhoramento constante do código, um processo chamado de refatoração. Esse procedimento visa melhorar a compreensão do código e, assim, reduzir a necessidade de documentação. Além disso, a refatoração também busca remover código redundante, para buscar uma re-estabilização da estrutura geral do sistema. Um código limpo e bem estruturado é mais fácil de alterar, quando não são necessárias alterações na sua arquitetura geral.

Na prática, nem todas estas sugestões são efetuadas em um projeto real. Como em todos os demais métodos, o XP costuma ser adaptado à realidade da equipe do projeto em particular.

2.5 Gerência de Projetos Ágeis

A principal responsabilidade do gerente é garantir que o projeto seja entregue dentro do prazo e orçamento previstos. Na abordagem dirigida a plano a gerência é relativamente simples, basta comparar o estado atual com o planejamento. Nas metodologias ágeis, no entanto, não há uma definição precisa de quando vais algo vai ser entregue ou quem vai trabalhar em cada etapa, dificultando a gerência.

Com isso, novas abordagens para gerenciamento são necessárias. Dentre elas, o método Scrum é, provavelmente, o mais utilizado. Ele é um método ágil geral, mas foca no gerenciamento de um desenvolvimento incremental.

2.5.1 Scrum

Esse *framework* para a organização e planejamento de projetos ágeis é dividido em três etapas.

Na fase inicial, são definidos em linhas gerais o objetivo do projeto e a arquitetura do *software* a ser utilizada. Esta etapa envolve o *Dev Team*, o *Scrum Master* e o *Product Owner*. Mas quem são essas pessoas?

O *Dev Team* é a equipe de desenvolvimento. No Scrum é um time pequeno e auto-organizado, de não mais de 8 pessoas, que são os responsáveis pela parte técnica, ou seja, escrever o código e a documentação. O *Scrum Master* é uma espécie de líder da equipe, ele garante que a metodologia está sendo seguida e faz a comunicação entre o a equipe, o cliente e o resto da empresa, protegendo-os de distrações externas. O *Product Owner* em geral é o cliente. É uma pessoa, ou grupo, que identifica e prioriza as funcionalidades e requisitos, além de verificar os *backlogs* para garantir que o projeto atende as necessidades. Mais uma palavra nova! O que é isso?

O *backlog* nada mais é do que a lista de coisas a fazer. Podem ser funcionalidades, requisitos, cenários ou tarefas extras, como definição de arquitetura e documentação.

A segunda etapa, que na realidade se repete várias vezes, é o desenvolvimento. Nesta metodologia é chamado de ciclo de *Sprint*, cada um destes ciclos faz um incremento no produto.

O ciclo de desenvolvimento é a unidade de planejamento. Eles geralmente duram de 2 a 4 semanas e iniciam com uma revisão das tarefas a fazer e definição de qual o próximo item a ser desenvolvido. A seleção envolve todo o time e normalmente leva em conta, além da prioridade, a velocidade do time, que é uma estimativa de quanto trabalho eles conseguem fazer em um ciclo.

Com todos de acordo, a equipe se organiza para desenvolver as tarefas escolhidas. Neste momento a equipe fica isolada do resto da empresa e do cliente. Diariamente ocorrem os *scrums*, que nada mais são que um encontro curto do time para verificar o progresso e definir os trabalhos a serem feitos neste dia. No fim do ciclo o trabalho é revisado e apresentado.

A última etapa da metodologia Scrum é o fechamento, quando o produto está satisfatório. Completa-se a documentação necessária, como ajudas e manuais, e entrega-se o produto ao cliente.

2.6 Qual Metodologia Utilizar?

Na prática, nenhum processo é certo ou errado, temos apenas aquele que mais se adequa à situação. Diversos fatores influenciam na decisão do método a ser seguido e quase sempre terminamos com um processo misto. Mas quais fatores são esses?

2.6.1 Fatores de Sistema

O **tamanho** do sistema é fundamental, métodos ágeis não funcionam muito bem em sistemas grandes. O **tipo** de sistema também, sistemas críticos necessitam de muita análise e um projeto detalhado antes de iniciar a implementação.

Outro ponto é a **expectativa de vida** do sistema. Sistemas de longa duração precisam de documentação para as equipes de suporte futuras. Por fim, pode haver alguma **regulamentação externa** que exige uma documentação detalhada.

2.6.2 Fatores Humanos

A **experiência** dos desenvolvedores influencia no projeto. Se o time é inexperiente, projetos detalhados facilitam o desenvolvimento. A **personalidade** dos membros da equipe também afeta a escolha, métodos ágeis necessitam de uma interação mais intensa.

Além disso, o **tamanho** e a **localização** do time podem fazer com que alguns documentos de projetos sejam necessários, é difícil organizar o trabalho sem projeto algum se as pessoas não estão no mesmo local, apesar de que com as facilidades de comunicação atuais este problema é bem menos importante que alguns anos atrás.

2.6.3 Fatores Organizacionais

Às vezes o método de desenvolvimento já está estabelecido na própria **cultura da empresa**. Em geral, nestes casos, é difícil de mudar. Grandes empresas e/ou antigas costumam ter uma cultura de documentação extensiva.

A **disponibilidade** e o **desejo** do **cliente** também pode inviabilizar o uso de métodos ágeis. O primeiro caso porque a presença constante do cliente é necessária para aplicação do método e o segundo porque alguns clientes querem uma documentação detalhada.

2.6.4 Mitos do Desenvolvimento

Seja lá qual for o método escolhido. É importante que não acreditemos em alguns mitos que são frequentemente repetidos na indústria de *software*. Alguns exemplos destes mitos são:

- *Basta seguir o método X.* Não, nenhum método é garantia de sucesso.
- *Se está atrasado, contrate mais gente.* Não, provavelmente isso vai atrasar ainda mais, fora o custo extra.
- *Se terceirizar, está resolvido o problema da gerência.* Não, gerenciar um projeto terceirizado é ainda mais complexo.
- *Basta uma ideia vaga para iniciar o projeto.* Não, isso provavelmente vai resultar em muito retrabalho depois.
- *Software é fácil de modificar.* Não mesmo, depende do tipo de modificação a ser feita.
- *O produto é o código.* Não, o código é parte do produto, quase sempre há uma documentação, nem que seja apenas de ajuda, e também o *design*.
- *ES só atrapalha.* Não, planejamento e documentação não são perda de tempo. Sempre são necessários, independentemente do tipo.

2.6.5 Princípios de Hooker

Em geral, utilizando ou não um método pré-estabelecida na literatura, podemos criar nossa própria metodologia bastando ter um pouco de bom senso. Hooker propõe sete princípios que são um ótimo guia para o desenvolvimento de *software*. Estes princípios são:

- **O *software* tem que ter uma razão de existir**

Senão, melhor não fazer. Isso quer dizer que é importante entender os requisitos reais e que sejamos capaz de agregar valor para o usuário.

- **Keep It Simple, Sir (KISS)**

O projeto deve ser o mais simples possível, mas não mais que isso. Fazer as coisas de forma simples em geral, toma mais tempo e raciocínio que fazer de outro jeito, mas tende a facilitar a manutenção e melhora a eficiência.

- **Mantenha o estilo**

Escolha um estilo para escrita do código e da documentação e garanta que este estilo é seguido por todos até o fim do projeto. Variações de estilo atrapalham a comunicação.

- **Outros consomem o que você produz**

Pense que outros vão ler e usar o que você escrever. Facilite a vida do outro e entregue a mesma qualidade que você exigiria. Pense nas consequências dos seus produtos para o cliente e a sociedade.

- **Esteja pronto para o futuro**

Pense na manutenção do seu produto, se você espera que ele tenha uma vida longa.

- **Planeja para reutilização**

Escrever funções genéricas configuráveis torna mais fácil a reutilização. Além disso, busque soluções já existentes.

- **Pense antes de fazer**

Avalie alternativas, verifique se não há uma forma melhor de fazer algo. Isso ajuda a reduzir os riscos de falha e retrabalho.

2.7 Recapitulando

- Projetos podem ser feito de forma sequencial, quando uma etapa só inicia depois que a anterior termina; ou de forma iterativa, onde as etapas são intercaladas.
- Em geral, os processos sequenciais são mais adequados para projetos grandes, com muitas pessoas envolvidas e que requerem projeto e documentação detalhada.

- Atualmente, a maioria dos *softwares* comerciais utiliza metodologias ágeis e um processo iterativo.
- Independente do tipo de processo, o reuso é uma ferramenta importante. Consiste em reutilizar códigos ou programas já prontos para resolver parte dos seus problemas.
- Atenção, é a ferramenta que deve se adequar ao problema e não o contrário.

Capítulo 3

Engenharia de Requisitos

Já vimos que a etapa inicial do desenvolvimento é a especificação. Essa etapa é crucial pois não há condições de construir alguma coisa sem saber “o que” ela é. A especialidade que trata do processo de especificação do *software* chama-se Engenharia de Requisitos.

Os requisitos são as descrições dos serviços e restrições do sistema a ser desenvolvido. Mesmo que o método seja ágil, é importante ter alguma documentação dos requisitos finais, como a coleção de cenário implementados, para facilitar o entendimento do sistema.

O nível de detalhamento da descrição do requisito depende de seu objetivo e de quem é o seu leitor. Podemos utilizar descrições mais abstratas e superficiais para uma conversa rápida com o usuário e uma ideia geral sobre o sistema e uma descrição detalhada para a etapa de projeto e uma definição mais precisa.

3.1 Tipos de Requisitos

Podemos dividir os requisitos em duas grandes classes: funcionais e não-funcionais. Em linhas gerais, os funcionais descrevem o que o sistema deve fazer e os não-funcionais são restrições, que muitas vezes se aplicam ao sistema como um todo, e não a uma tarefa específica. Vamos ver um pouco mais de detalhes sobre cada um.

3.1.1 Requisitos funcionais

Descrevem aquilo que o sistema deve fazer, e às vezes o que não deve fazer também. Essa descrição deve ser precisa, pois uma comunicação ambígua pode levar a interpretação distintas do usuário e dos desenvolvedores.

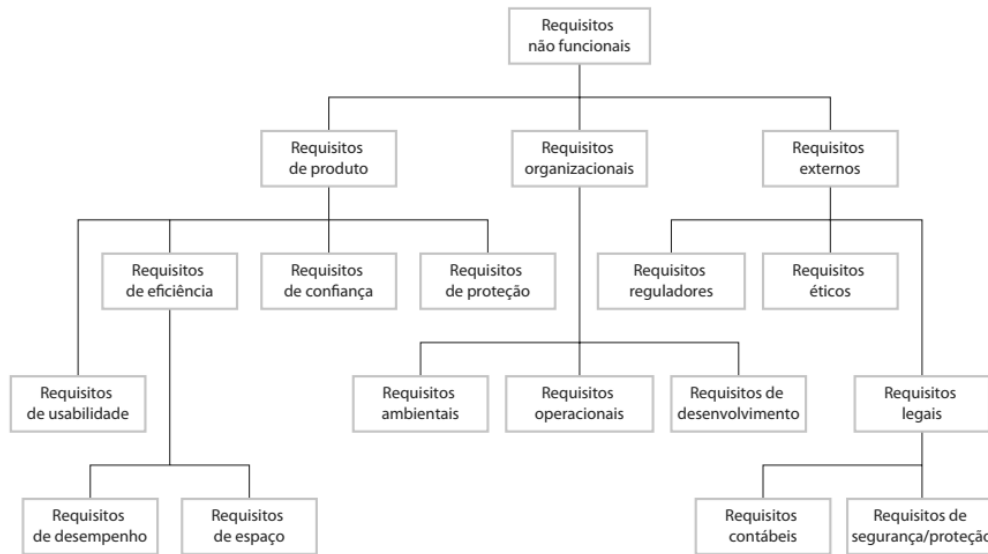


Figura 3.1: Tipos de requisito não-funcional

Idealmente, o conjunto de requisitos funcionais deve ser completo e consistente. Completo, no sentido de descrever todas as funcionalidades que o cliente necessita; e consistente, no sentido de que não há conflitos entre os requisitos. Na prática, isso quase nunca ocorre.

Em sistemas muito grandes, é fácil haver erros ou omissões, em sistemas pequenos, como normalmente é utilizada uma metodologia ágil, é fácil haver omissões. Além disso, um sistema costuma ter vários *stakeholders* (pessoas que são afetadas e estão interessadas no sistema) com necessidades e prioridades distintas. Essa distinção muitas vezes leva a inconsistências que podem passar despercebidas.

3.1.2 Requisitos não-funcionais

Os não-funcionais são as propriedades e restrições do sistema. Às vezes até requisitos do próprio processo de desenvolvimento ocorrem, como linguagem de programação a ser utilizada. Os requisitos não-funcionais são critérios como confiabilidade, eficiência, segurança, e por isso costumam ser mais abrangentes e mais críticos que os requisitos funcionais. Às vezes um único requisito não-funcional gera diversos funcionais relacionados para atendê-lo. Na figura 3.1 temos alguns tipos de requisitos não funcionais.

Por ser mais qualitativo, muitas vezes é difícil enunciar este tipo de requisito de forma precisa. Precisamos transformar o objetivo do cliente, ou seja,

sua intenção, em um requisito verificável, que tenha um critério que pode ser objetivamente testado.

Devemos trocar termos como “rápido” por um valor específico de tempo de resposta e “confiável” por alguma métrica específica como disponibilidade, probabilidade de erro, etc.

3.2 O Processo de Requisitos

O processo da engenharia de requisitos costuma começar com um estudo de viabilidade. Essa etapa busca uma estimativa da possibilidade técnica se satisfazer as necessidades do cliente usando as tecnologias atuais e também se o sistema é rentável para o desenvolvedor. Essa etapa deve ser rápida e barata, é ela quem determina se o projeto vai iniciar.

Em seguida, entra a elicitação e análise de requisitos, que é a parte central do processo, onde os requisitos são descobertos, classificados, priorizados e eventualmente negociados com o cliente, caso necessário.

Com os requisitos definidos, temos a especificação de requisitos, que é o documento com o detalhamento dos mesmos. De posse deste documento, faz-se uma validação dos requisitos para verificar que atendem às necessidades do cliente.

3.2.1 Elicitação e Análise

Aqui é onde os requisitos são descobertos. É um trabalho conjunto da equipe técnica com o cliente, para entender o domínio da aplicação, os serviços que o sistema deve prover e as restrições operacionais. Pode envolver todos os *stakeholders*, usuários, gerentes, engenheiros de manutenção, etc.

Os principais problemas nesta etapa inicial de descoberta é que nem sempre os *stakeholders* sabem o que realmente querem, além de se expressarem em seu próprio jargão. Além disso, *stakeholders* distintos podem ter requisitos conflitantes que precisam ser negociados ou priorizados. Os requisitos também podem depender de políticas organizacionais da empresa desenvolvedora ou do cliente.

Como é um processo de descoberta, de conversa, os requisitos costumam mudar bastante nas primeiras iterações de elicitação e análise. Também na fase de descoberta investigamos sistemas existentes para problemas similares ou subproblemas.

A forma mais comum de fazer o processo de elicitação é através de entrevistas. As entrevistas podem ser formais ou informais, fechadas, baseadas

em questões pré-determinadas, ou abertas, sem questões pré-definidas com o objetivo de dar mais liberdade para exploração.

É sempre importante registrar a entrevista para referência futura. Para ser eficiente, é necessário estar com a mente aberta, disposto a ouvir e sem ideias pré-concebidas acerca do sistema. O ideal é provocar o entrevistado, fazer ele continuar falando usando perguntas chaves, propostas de requerimentos ou mesmo trabalhar juntos num protótipo, que pode ser até de papel e lápis.

A grande dificuldade com as entrevistas é a comunicação. Especialistas em um domínio podem usar uma linguagem para descrever seu trabalho que é incompreensível para quem está fora da área. Além disso, pontos considerados “óbvios” pela pessoa entrevistada costumam ser omitidos, já que ela acredita que todos sabem, e, quando expressos, normalmente são mal articulados.

Uma abordagem alternativa à entrevista é a etnografia. Que consiste na observação do trabalho do jeito que ele realmente é feito. Deste modo, o observador é capaz de descobrir requisitos reais que, às vezes, nem os próprios trabalhadores sabiam. Normalmente são requisitos que emergem da cooperação e/ou competição no ambiente. Além disso, o trabalho real nem sempre é feito como deveria ser feito segundo alguma norma.

A etnografia é capaz de entender melhor como se dá o processo de trabalho atual, mas não é adequada para descobrir características novas para serem acrescentadas ao sistemas.

Por fim, a construção de cenários, a técnica usada na XP, costuma ser uma boa forma de comunicação. Em geral, é mais fácil para as pessoas darem um exemplo de uma situação real do que descrições abstratas.

Nestas situações elas estão mais dispostas a debater. É justamente deste debate que podemos detalhar os requisitos. Um cenário é uma descrição estruturada de uma interação com o sistema, que deve incluir uma descrição da situação inicial do sistema, como seria um fluxo normal de eventos, o que pode dar errado, informações sobre atividades concorrentes e uma descrição do sistema ao fim do cenário.

Com os requisitos definidos, vamos à redação.

3.2.2 Especificação

Esta etapa é a escrita do documento de requisitos, muitas vezes temos uma versão mais detalhada, para a equipe de desenvolvimento e uma versão mais simples para os demais *stakeholders*. Idealmente, os requisitos devem dizer apenas “o que” deve ser feito, e não “como” é feito, esta é a etapa de Projeto.

Na prática essas duas etapas são inseparáveis, às vezes os requisitos, em especial os não-funcionais, afetam diretamente o projeto. Seja para atender uma demanda de arquitetura, uma necessidade de comunicação com outros sistemas ou regulamentações externas.

Os requisitos podem ser expressos de diversas formas. A linguagem natural é a mais fácil de ser entendida por todos os envolvidos. Ao mesmo tempo, é também a mais facilmente mal interpretada por causa da ambiguidade da linguagem. Muitas vezes é difícil ser claro sem deixar a frase muito longa.

Quando escrevemos qualquer documento, precisamos seguir algum padrão, a consistência ajuda na aprendizagem. Outra característica que ajuda a leitura é o destaque visual de partes essenciais. O uso de elementos de destaque também deve ser consistente. Também é fundamental usar a língua de forma correta, tanto pela sua imagem e da empresa, quanto pela necessidade de ser preciso.

Sempre que possível evite o uso de jargão técnico. Se isso for realmente necessário, faça duas versões da redação, uma simplificada e outra técnica. Por fim, sempre inclua uma explicação de porque o requisito é necessário.

Devemos tomar cuidado para não confundir requisitos funcionais e não-funcionais e também para não agrupar vários requisitos distintos em um só.

Muitas vezes os projetos utilizam formulários de especificação, com uma descrição da entidade, as entradas e suas origens, as saídas e seus destinos. Descreve-se a ação a ser realizada e o que precisa ser necessário antes e após a realização da tarefa. Isso ajuda a padronizar o documento, e são particularmente úteis para especificações funcionais.

Uma forma cada vez mais comum de descrever os requisitos é através dos diagramas de casos de uso, que essencialmente é a mesma ideia dos cenários da XP. Utiliza-se alguma linguagem de modelagem (quase sempre a UML) para representar e descrever as possíveis interações com o sistema. O diagrama é bem simples, os atores, ou seja, aqueles que fazem a ação, são representados por bonequinhos “palito” enquanto os tipos de interação são representados por elipses. Os atores e interações são unidos por setas, onde a direção da flecha indica quem inicia a ação. Cada caso de uso é acompanhado de uma descrição textual, explicando a interação.

Apesar de serem chamados de “atores” o nome do caso de uso deve sempre lembrar o papel dele na interação e não o nome do ator. Além das relações entre atores e casos de uso, chamadas de associações. Podemos ter relações entre atores no sentido de generalizações, por exemplo, “Usuário” generaliza “Morador” e “Visitante” em um sistema de controle de um condomínio. No diagrama, a seta aponta para o ator genérico.

Além disso, também podem haver relações entre os casos de uso. As relações de inclusão indicam uma relação obrigatória, por exemplo, a operação

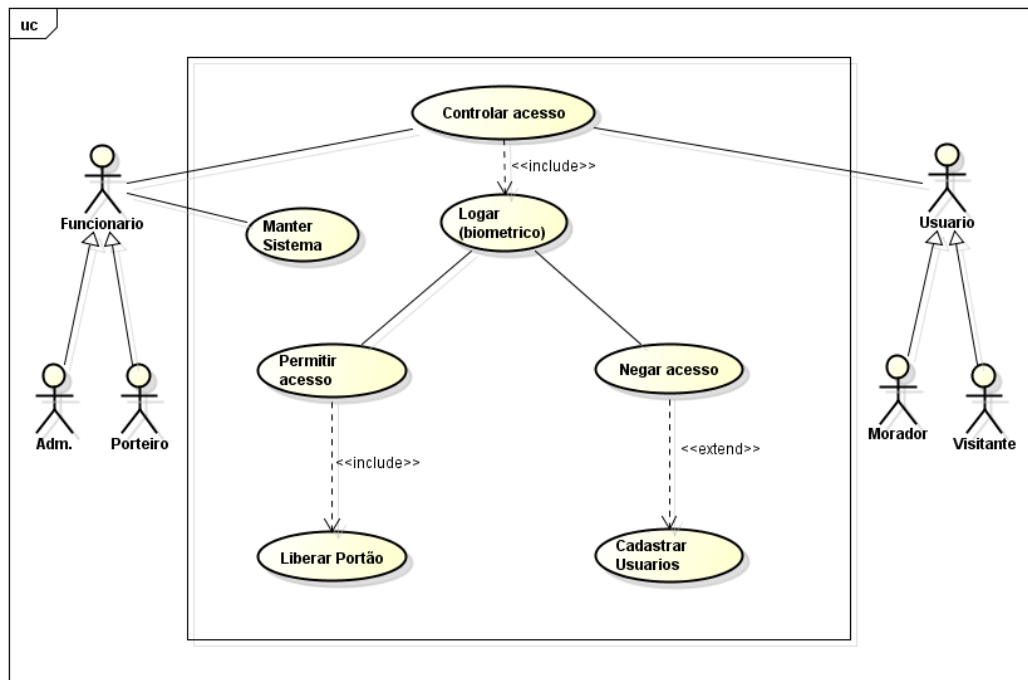


Figura 3.2: Exemplo de diagrama UML de caso de uso

“Permitir Acesso” inclui a operação “Liberar Portão”. Já as relações de extensão são opcionais, “Negar Acesso” pode ser estendida por “Cadastrar Usuário”, por exemplo. No diagrama a seta aponta para o caso de uso que está sendo incluído ou estendido. Temos um exemplo simples na figura 3.2.

3.2.3 Validação

Nesta etapa o documento de requisitos é utilizado para garantir que os requisitos elicitados definem o que o cliente realmente quer. Corrigir um erro de requisito após a entrega custa dezenas, ou até centenas de vezes mais caro que corrigir um erro de implementação.

O que precisamos verificar na etapa de validação dos requisitos? Os principais pontos são os seguintes:

- O sistema fornece as funções que melhor atendem as necessidades do cliente?
- Existem requisitos em conflitos?
- Todas as funcionalidades solicitadas estão incluídas?

- As funcionalidades podem ser implementadas com o orçamento e tecnologia disponíveis?
- Os requisitos podem ser testados?

A validação pode ser feita através de uma análise manual dos requisitos, utilizando um protótipo ou através de casos de teste.

Como sempre, uma boa comunicação entre *stakeholders* e desenvolvedores é fundamental para identificar e resolver os problemas no início.

As etapas de elicitación, especificação e validação são intercaladas, podendo levar algumas iterações até chegar numa especificação adequada do sistema.

3.3 Gerenciamento de Requisitos

Já falamos diversas vezes, os requisitos sempre mudam. Nem sempre por erro, simplesmente a realidade muda. Além disso, os *stakeholders* são variados, é bem comum que tenhamos requisitos conflitantes.

É necessário gerenciar esse dinamismo dos requisitos durante o desenvolvimento. Para isso, é importante ser capaz de rastrear os requisitos individuais e manter conexões entre requisitos dependentes.

Outra coisa que ajuda na gerência é ter um processo formal para propor mudanças. A ideia é que quem quer a mudança faz a proposta e ela é primeiro analisada para verificar se o problema de fato existe e se a alteração proposta resolve o problema. Se não passar nessa etapa, devolve-se o pedido. Supondo que a alteração seja válida, devemos avaliar o custo da modificação, verificar o impacto dela nos demais requisitos e no sistema como um todo. Se for inviável, descarta-se a modificação. Se a modificação é válida e viável, altera-se o documento de requisitos, o projeto do sistema e a sua implementação.

3.4 Recapitulando

- O documento de requisitos é fundamental.
- Requisitos funcionais descrevem funcionalidades (tarefas, ações) enquanto requisitos não-funcionais descrevem características ou restrições, quase sempre mais genéricas.
- Entrevistas com os *stakeholders* são a forma mais comum de descobrir os requisitos.

- Protótipos e casos de uso podem ajudar na entrevista.
- O documento de projeto pode ter diversos níveis de detalhamento, geralmente temos um nível mais superficial para os usuários e um mais detalhado para os desenvolvedores.
- É bastante comum o uso de diagramas de caso de uso da UML.
- O documento deve ser validado por todos.
- O processo de requisito deve ser constante, porque é muito mais barato alterar um requisito durante o desenvolvimento que após a entrega.

Capítulo 4

Projeto de *Software*

Antes de iniciar um detalhamento de como é feita a modelagem no projeto de *software*, vamos rever alguns conceitos fundamentais da orientação a objetos.

4.1 Orientação a Objetos (OO)

Há diversos paradigmas para a implementação de um *software*, mas os dois mais utilizados são o paradigma imperativo e o paradigma orientado a objetos (OO).

No paradigma imperativo a ideia geral é o uso de subprogramação, que nada mais é que a utilização de uma sequência de atividades simples para gerar uma atividade complexa.

Esses blocos de atividades podem ser nomeados e normalmente são chamados de procedimento, subrotina ou função. Em qualquer ponto do código é possível chamar uma atividade complexa pelo nome, ao invés de ter de reescrever todo o código.

Um conjunto de subrotinas pode ser agrupado em uma biblioteca.

No paradigma OO, a ideia é representar os elementos do sistema como Objetos e agrupá-los em classes de objetos, que possuem características semelhantes, e ainda agrupar classes similares em pacotes.

O mundo é composto de objetos que se relacionam, a ideia central da orientação a objetos é criar modelos computacionais dos objetos do mundo real e das suas relações. Portanto, um objeto é uma representação de um elemento ou processo do mundo real e, como tal, possui suas características e comportamentos.

Uma característica, chamada de Atributo no jargão da área, descreve uma propriedade, ou seja, um elemento que compõe o objeto. Computacionalmente, os atributos são variáveis que pertencem ao objeto.

Um comportamento, chamado de Método, é uma ação ou resposta de um objeto a uma ação do mundo real. Computacionalmente é equivalente às funções da programação imperativa e também pertencem ao objeto ou à classe.

Há quatro princípios que são utilizados na programação orientada a objetos:

- Abstração
- Encapsulamento
- Modularidade
- Hierarquia

A abstração diz que o objeto computacional deve se concentrar nos atributos e métodos que são relevantes para o problema que está sendo tratado. Em diferentes contextos um mesmo objeto pode ter atributos e métodos distintos.

O encapsulamento diz que os objetos devem esconder seus dados e detalhes de implementação. Os métodos formam uma “cerca” em torno dos atributos, que devem ser manipulados apenas através de métodos.

Alguns métodos são públicos, podendo ser utilizados por outros objetos através de uma chamada e é a forma de comunicação entre objetos. Outros métodos são privados, que cuidam e só podem ser utilizados pelo próprio objeto. Há ainda os métodos estáticos, que são chamadas diretamente da Classe e não do Objeto.

A modularidade consiste em dividir o todo em partes bem definidas, que podem ser construídas e examinadas separadamente. Essas partes interagem entre si, fazendo o sistema funcionar de forma adequada. Um programa em OO é um conjunto de objetos que colaboram entre si, através da chamada de métodos, para resolver o problema.

Os objetos possuem uma hierarquia, herdando atributos e métodos de seus ancestrais. A herança é um mecanismo para simplificar a definição de classes de objetos similares.

Bom, mas o que é uma Classe? Classe é o tipo do objeto, é uma espécie de forma que cria cópias, instâncias no jargão da área, do objeto. Nós criamos as Classes e as classes criam os objetos. As classes determinam os atributos e comportamentos comuns a todos os objetos do mesmo tipo.

Para viabilizar a hierarquia de objetos, as classes possuem um mecanismo de herança. A classe que fornece elementos, a antecessora, é chamada de superclasse; e a classe que recebe elementos, a herdeira, é chamada de subclasse.

A subclasse pode herdar elementos de várias superclasses e pode ter métodos e atributos próprios, diferentes da classe.

Além das classes e objetos, há dois tipos de classe especiais, nem sempre disponíveis em toda linguagem orientada a objeto, as interfaces e as classes abstratas.

As interfaces são classes que não têm implementação, seus atributos são constantes e seus métodos possuem apenas a assinatura, ou seja, definição de entradas e saídas. Uma classe que implementa uma interface deve implementar seus métodos. Podemos pensar na interface como um contrato assinado pela classe que a implementa de que ela fornecerá certos métodos.

As classes abstratas não podem ser instanciadas, definem atributos e métodos, que podem ser comuns ou abstratos. Se não pode ser instanciada, serve pra que? Ela serve para definir a estrutura das subclasses e para implementar uma característica chamada polimorfismo. O polimorfismo ocorre quando uma subclasse redefine um método herdado. Assim o mesmo método tem implementações distintas a depender da subclasse que você utiliza.

4.2 Especificação x Projeto

A orientação a objetos facilita o processo de projeto a partir dos casos de uso. Ela diminui a distância entre as fases de projeto e desenvolvimento.

Não há um consenso acerca de onde termina a especificação e inicia o projeto, a grande diferença entre as duas etapas está na ênfase que é dada. Na especificação devemos focar em entender o problema, o que deve ser feito. No projeto devemos focar em como implementar o que foi decidido na especificação.

As duas etapas são representadas por modelos, já vimos os casos de uso para os requisitos, que são abstrações que servem para ilustrar os elementos e relacionamentos entre as entidades e funcionalidades do sistema e do ambiente que o cerca.

4.3 Alguns Tipos de Modelo

Um modelo deve focar no que interessa para um determinado observador em um dado momento. Servem para facilitar a comunicação, lidar com problemas complexos e testar hipóteses antes de realizar. Todos os domínios do conhecimento usam modelos.

Mas cuidado, se o seu modelo não serve para as atividades citadas no parágrafo anterior, talvez seja melhor descartá-lo.

O modelo é o conteúdo, ele representa o problema ou a solução que está sendo discutida. Um diagrama é uma visualização do modelo ou de parte dele. Assim como temos diversos modelos, há também diversas formas de visualização.

4.3.1 Modelos de Contexto

Normalmente este tipo de utilizado na etapa inicial, na análise de requisitos. O seu principal objetivo é representar o contexto operacional geral, definir o que é o sistema e o que é o ambiente, ou seja, o que é externo ao sistema. Já fizemos isso nos casos de uso.

As fronteiras do sistema podem ser afetadas, além do desejo dos clientes e usuários, de preocupações sociais e organizacionais. Por exemplo, numa situação onde um sistema da empresa tem restrições de segurança ou simplesmente queremos nos livrar da necessidade de autorizações de outros departamentos.

Após as decisões do limite do sistema, podemos produzir um diagrama simples da arquitetura, que mostra o sistema e sua relação com outros sistemas.

4.3.2 Modelos de Processo

Os diagramas de arquitetura não mostram as relações entre os sistemas e o ambiente. Sistemas externos podem produzir ou consumir dados do sistema. Quando completamos o modelo de arquitetura com informações sobre os processos do sistema e aqueles suportado por ele temos um modelo de processo.

Nos diagramas de processo temos áreas destinadas aos atores e sistemas e quais os processos realizados por cada um deles. Normalmente são utilizadas setas para indicar a direção do fluxo.

Um exemplo de diagrama de processo pode ser visto na figura 4.1. Neste tipo de diagrama costumamos utilizar círculos para indicar pontos de entrada e saída do processo. Os procedimentos são retângulos de borda arredondada e os losangos representam decisões. Os retângulos representam informações extras como, ferramentas ou documentos necessários para a ação.

4.3.3 Modelos Comportamentais

Como o nome diz, estes modelos representam o comportamento geral do sistema. Os principais tipos de diagramas utilizados nestes modelos são os fluxos de dados e as máquinas de estado.

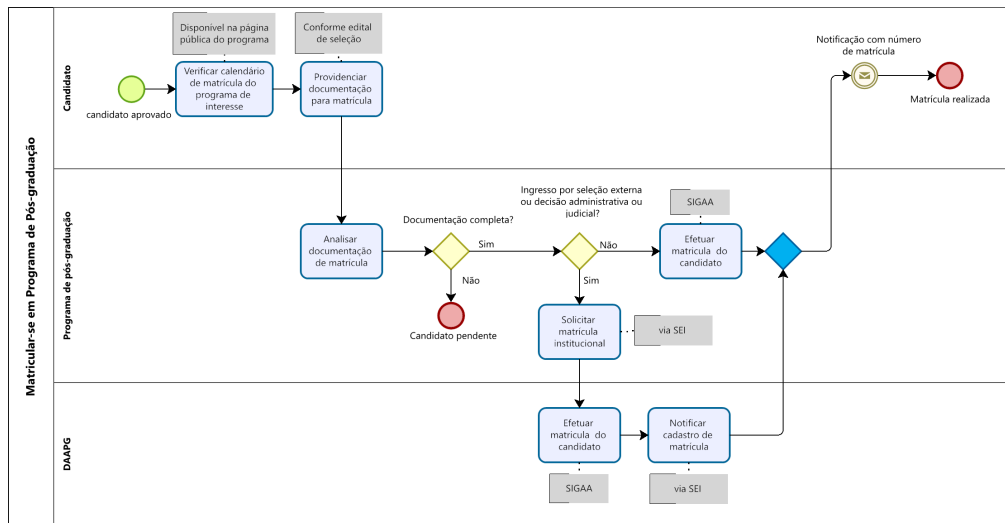


Figura 4.1: Exemplo de diagrama de processo

Nos diagramas de fluxo de dados representa-se como os dados são processados e como se movem através do sistema. Já nos diagramas de máquinas de estado é mostrado como o sistema responde a eventos internos ou externos.

Na figura 4.2 temos um exemplo genérico de diagrama de fluxo de dados. Nestes diagramas os processos são representados por retângulos arredondados e as entidades externas por retângulos. O fluxo de dados é representado por setas que são rotuladas com o dado que é transferido de um processo para o outro. Por fim, utilizamos retângulos com um quadrado rotulado à esquerda para indicar depósitos de dados, como bancos de dados ou arquivos.

Perceba que esse diagrama é bastante semelhante ao de processo e muitas vezes um é usado no lugar do outro.

Na figura 4.3 temos um exemplo de diagrama de máquinas de estados. Neste tipo de diagrama assumimos que o sistema está em um estado, ou seja, numa situação definida por suas variáveis e programação e só muda de estado em resposta a algum evento. Um evento nada mais é que uma ação, pode ser o pressionar de um botão, um tempo que acabou, qualquer coisa que faça o sistema sofrer uma mudança de estado.

A cada instante o sistema deve estar em um, e somente um estado. Ao entrar num estado é possível executar uma ou mais ações. Outro ponto importante é que o evento não precisa ser uma ação externa, pode ser um

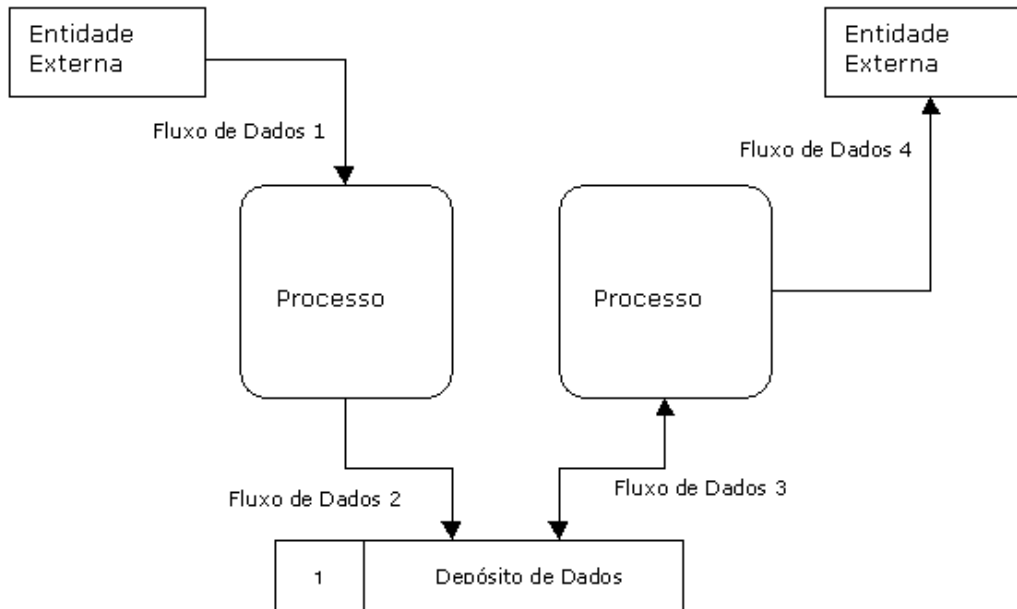


Figura 4.2: Exemplo de diagrama de fluxo de dados

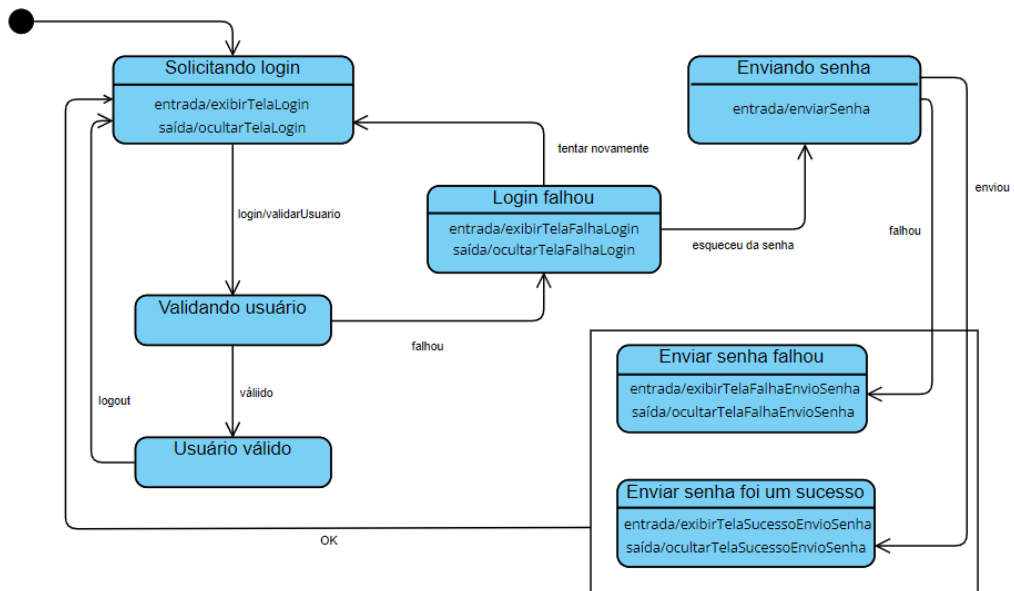


Figura 4.3: Exemplo de diagrama máquina de estados

evento interno ao sistema, como descrito no parágrafo anterior.

Dependendo do nível de detalhamento e da complexidade do sistema o número de estados pode ser muito grande. Neste caso costumamos criar superestados que, por sua vez, podem ser detalhados em um diagrama próprio.

Em geral são utilizados retângulos arredondados para os estados e setas para as transições, que são rotuladas pelo evento que as provocou. Também é comum a utilização de pontos de início e fim, representados por círculos.

4.3.4 Modelo de Dados

O objeto deste modelo é descrever a estrutura lógica dos dados. A técnica mais utilizada é chamada de Entidade - Relacionamento - Atributo (ERA) que, como o nome sugere, define as entidades do sistema, os relacionamentos entre estas entidades e os atributos que cada uma possui. O uso mais comum é na descrição de bancos de dados, que veremos com mais detalhes no capítulo 5.

Um exemplo de modelo de dados é apresentado na figura 4.4. Os retângulos são as entidades, que possuem uma lista de atributos, por exemplo o Curso tem um código, uma descrição e uma carga horária. Entre as entidades temos conexões que representam o tipo de relacionamento entre elas. No diagrama do exemplo, quando o losango é branco indica que é “um” e quando é preto indica que são “vários”. Por exemplo, um professor leciona várias turmas. Um curso é coordenado por um professor, e assim por diante.

Além dos diagramas ERA, é também comum termos dicionários de dados, que nada mais são que uma lista de todos os nomes usados nos modelos do sistema. Muitas vezes uma descrição das entidades, dos atributos e dos relacionamentos também são incluídos.

O dicionário de dados serve como um repositório de informações, ligando as etapas do processo de desenvolvimento. Além disso, elas evitam a utilização de nomes duplicados.

4.3.5 Modelos de Objetos

Este modelo descreve o sistema em termos das suas classes e das associações entre estas classes. Num modelo de objetos temos diversos variantes como hierarquia, agregação e comportamento.

Os diagramas de classe representam estes modelos. Quase sempre as classes são representadas por retângulos com o nome da classe na parte superior, os atributos na seção intermediária e os métodos na seção inferior. Os relacionamentos, ou associações, são representadas por linhas que ligam as classes.

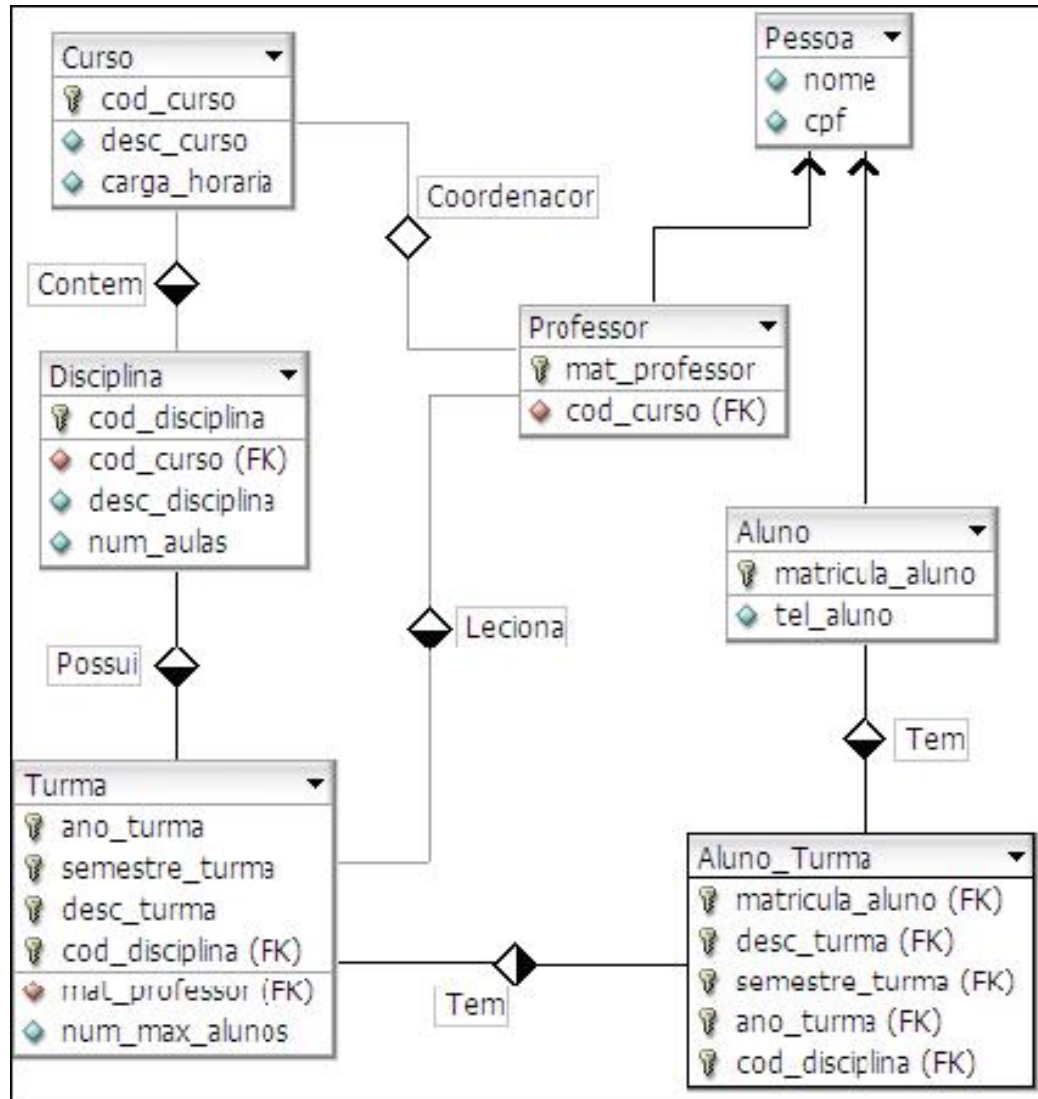


Figura 4.4: Exemplo de model de dados

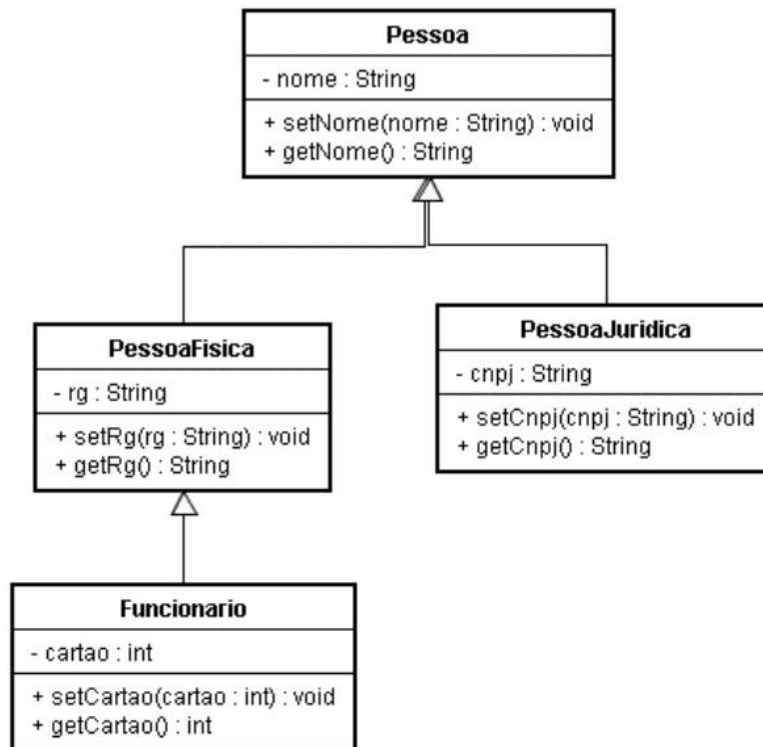


Figura 4.5: Exemplo de diagrama de hierarquia de classes

No modelo de hierarquia de objetos temos uma descrição da hierarquia das classes. As classes que recebem uma seta são superclasses enquanto aquelas que enviam as setas são subclasses. Na orientação a objetos normalmente pensamos em herança, aqui o termo utilizado é “generalização”, mas a ideia é a mesma. A subclasse herda métodos e atributos das superclasses e pode ser especializada se necessário. Temos um exemplo deste tipo de diagrama na figura 4.5.

Alguns sistemas permitem heranças múltiplas, onde uma classe herda de várias superclasses. Isso deve ser utilizado com cuidado, pois pode levar a conflitos semânticos se atributos ou métodos com o mesmo nome em superclasses distintas têm significados distintos.

No modelo de agregação é mostrado como as coleções de classes são compostas. Normalmente é representado por uma seta com um losango numa ponta e pode ser lida como “é parte de”. Este tipo de relacionamento também pode ocorrer em outros tipos de diagramas, como diagramas ERA ou fluxos de dados. Na figura 4.6 temos um exemplo de agregação. Neste exemplo, Tarefa, Resumo, Apresentação e Aula são partes de Curso.

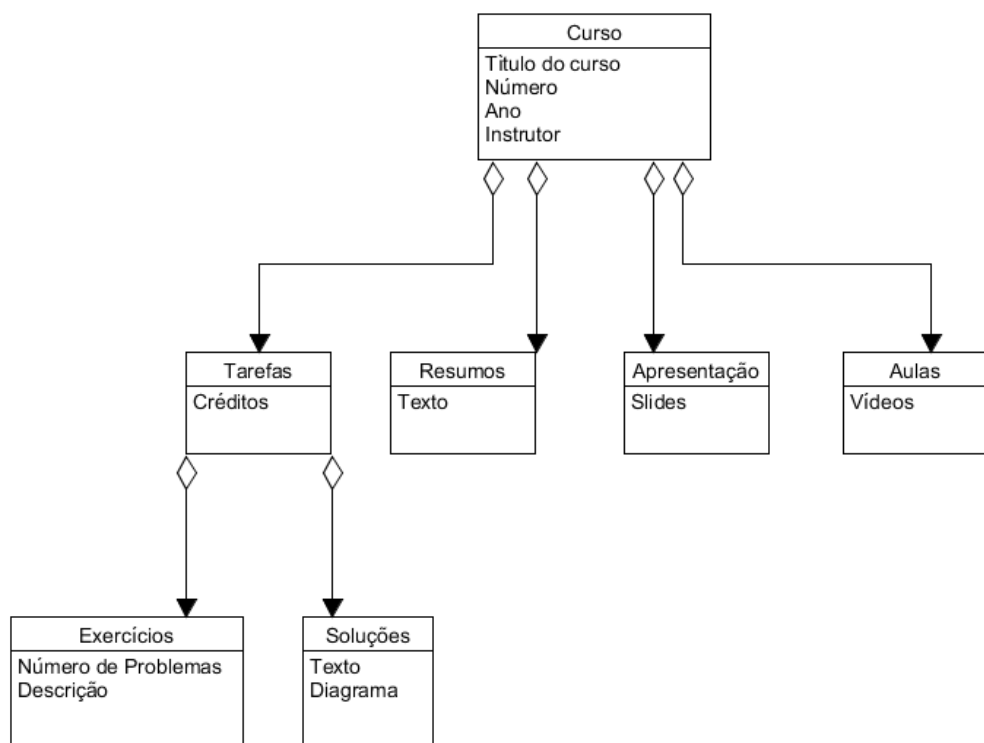


Figura 4.6: Exemplo de diagrama agregação de classes

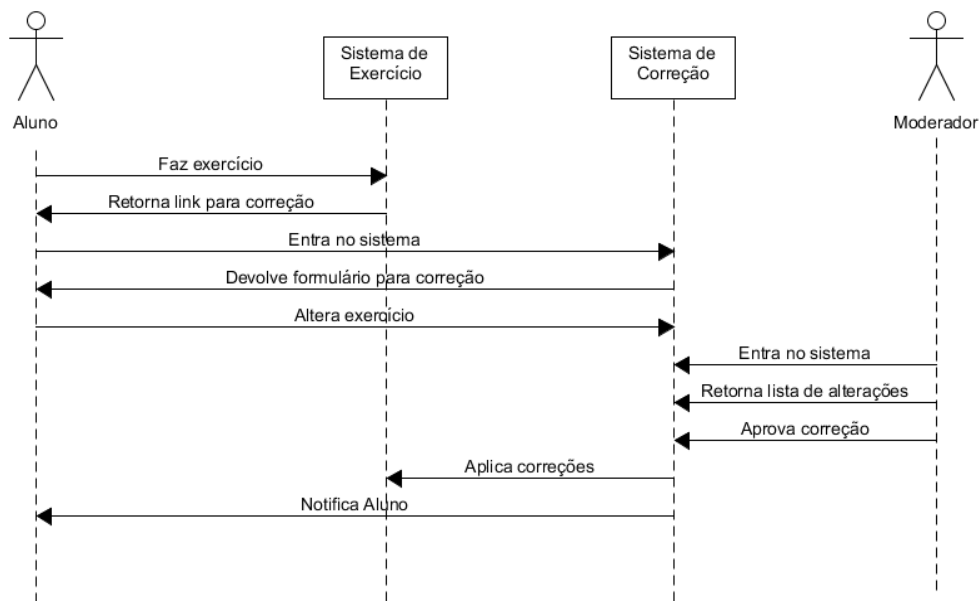


Figura 4.7: Exemplo de diagrama de sequência

Por fim, nos modelos de comportamento, temos um detalhamento das interações entre objetos e agentes externos. Os diagramas de sequência, ou de colaboração, são geralmente utilizados para modelar a interação entre objetos.

Neste diagrama as classes ou subsistemas são representados por retângulos e os atores por bonecos palito. A sequência, de cima para baixo indica o tempo. As setas na horizontal indicam quem realiza a ação e quem recebe o resultado dessa ação.

Capítulo 5

CRUD

No universo da programação e da engenharia de software, o termo CRUD é uma das primeiras siglas que um desenvolvedor aprende.

Ela representa as operações *Create* (criar), *Read* (ler), *Update* (atualizar) e *Delete* (apagar), o CRUD é a base do uso de dados em sistemas de informação.

A operação *create* refere-se à inserção de novos dados, a operação *read* refere-se à consulta e leitura de dados inseridos, *update* refere-se à modificação de dados existentes e *delete* à remoção de dados cadastrados.

Essas operações são fundamentais para qualquer aplicação que manipule informações dinâmicas, seja um comércio, uma planilha, um jogo ou uma rede social.

Além de representar a manipulação de dados, o CRUD é um modelo conceitual que o guia o design de interfaces de usuário, lógicas de negócio e estruturação de bancos de dados.

Um CRUD implementado corretamente garante um fluxo lógico consistente, uma melhor experiência para o usuário e facilidade na manutenção e escalabilidade do sistema.

Ao implementar um sistema deste tipo devemos ter atenção a alguns aspectos como segurança, prevenindo a possibilidade de ataques; performance, otimizando consultas; validação de dados, garantindo que os dados inseridos sejam corretos e completos; e consistência, para evitar a corrupção dos dados.

Aprender a desenvolver um CRUD é essencial para a carreira de um desenvolvedor de *software*. Conhecimento prático deste assunto é cobrado em entrevistas técnicas, testes de lógica de programação e é a base de muitos projetos de *software*.

Ao longo deste capítulo vamos utilizar como base o CRUD e os conhecimentos que vocês já têm em programação e engenharia de *software* para introduzir alguns conceitos novos. Ao fim, espero que tenhamos um CRUD

simples implementado.

5.1 Um CRUD simples

Imagine que temos um sistema onde, em parte dele, precisamos cadastrar um conjunto de jogadores. As informações que precisam ser registradas acerca dos jogadores são: nome, idade e sexo.

Como não sabemos quantos jogadores são, podemos utilizar uma lista para guardar os mesmos, onde cada elemento é um jogador. Um jogador individual pode ser representado por uma lista, onde cada posição corresponde a uma informação e, neste caso, precisamos memorizar ou registrar qual posição corresponde a cada informação; ou por um dicionário, que no fim das contas podemos entender como uma espécie de lista onde as posições têm nomes ao invés de números.

Vamos fazer um pequeno aplicativo CRUD correspondente à esta parte do sistema. Nosso aplicativo deve exibir as opções CRUD e o usuário deve selecionar e realizar as operações desejadas. Além disso, o programa deve rodar indefinidamente até que o usuário deseje encerrar. A estrutura geral deste programa pode ser vista na listagem 5.1.

```
1 //inicializa variáveis
2
3 while (continuar) {
4     //exibe menu de opções
5
6     //obtem opção do usuário
7
8     switch(opcao) {
9         case '0':
10             //comandos para sair da aplicação
11             break
12         case '1':
13             //comandos para adicionar jogador (create)
14             break
15         case '2':
16             //comandos para listar jogadores (read)
17             break
18         case '3':
19             //comandos para editar jogador (update)
20             break
21         case '4':
22             //comandos para apagar jogador (delete)
23             break
24     }
```

25 }

Listing 5.1: Esqueleto do CRUD

Nós poderíamos começar a popular esta estrutura com os códigos necessários, mas vamos parar pra pensar um pouco antes de começar.

Tanto na momento de apagar quanto no momento de editar, seria interessante listar os jogadores para que o usuário possa escolher qual vai ser editado ou removido. Se escrevemos um código monolítico, sem utilizar funções, vamos precisar reescrever o código da listagem várias vezes. Isso implica que, toda vez que decidirmos alterar a forma da listagem, teremos vários pontos do código que precisam ser atualizados.

Ao invés disso, vamos tentar utilizar funções para separar o trabalho e evitar a reescrita constante de códigos. A estrutura na listagem 5.2 é uma proposta de como podemos implementar.

```
1  /*Pode ser necessário instalar o módulo prompt-sync com o
   comando: npm install prompt-sync*/
2
3  const prompt = require('prompt-sync')()
4
5  const cadastro = []
6  var continuar = true
7
8  while (continuar) {
9      const opcao = menu()
10
11      switch(opcao) {
12          case '0':
13              sair()
14              break
15          case '1':
16              adicionarJogador()
17              break
18          case '2':
19              if (cadastro.length == 0){
20                  cadastroVazio()
21                  break
22              }
23              listarJogadores()
24              break
25          case '3':
26              if (cadastro.length == 0){
27                  cadastroVazio()
28                  break
29              }
30              editarJogador()
31              break
```

```

32     case '4':
33         if (cadastro.length == 0){
34             cadastroVazio()
35             break
36         }
37         apagarJogador()
38         break
39     default:
40         opcaoInvalida()
41         break
42 }
43 }

```

Listing 5.2: Estrutura geral do nosso CRUD

Nesta nova estrutura eu incluí o caso `default` no `switch` para tratar o caso de opções inválidas. A variável `cadastro` é uma lista, inicialmente vazia, que vai guardar nossos jogadores. A variável `continuar` determina se o programa continua sendo executado ou não.

Além disso, as opções de listar, editar e apagar só devem ser executadas se o cadastro não estiver vazio. A função `cadastroVazio()` apenas exibe uma mensagem de que o cadastro está vazio.

Da forma que foi escrito, a função `menu` deve exibir as opções e retornar a opção feita pelo usuário, de tal modo que 0 é sair, 1 é adicionar, 2 é listar, 3 é atualizar, 4 é apagar e todas as outras opções são inválidas.

Vamos começar implementando a função de exibição do menu na listagem 5.3. Ela simplesmente apaga a tela, exibe as opções numeradas e pede que o usuário escolha uma delas, depois a opção do usuário é retornada e a tela é apagada. Perceba que a barra invertida é digitada duas vezes, porque esse caractere é utilizado para alguns comandos especiais da linguagem e a barra invertida duplicada é exibida como uma barra única.

```

1  function menu() {
2      console.clear()
3      console.log("/=====\\")
4      console.log("|      MENU      |")
5      console.log("\\=====/")
6      console.log("| 1. Adicionar  |")
7      console.log("| 2. Listar    |")
8      console.log("| 3. Editar    |")
9      console.log("| 4. Apagar    |")
10     console.log("|-----|")
11     console.log("| 0. Sair      |")
12     console.log("\\=====/")
13     console.log()
14
15     const opcao = prompt("Selecione uma opção: ")

```

```
16 console.clear()
17 return opcao
18 }
```

Listing 5.3: Função para exibir menu

Nossa próxima função é de saída, ela deve simplesmente exibir uma mensagem e alterar o valor da variável `continuar` para que o laço `while` pare. Uma possibilidade está na listagem 5.4

```
1 function sair() {
2   console.log("Saindo do aplicativo...")
3   console.log()
4   continuar = false
5 }
```

Listing 5.4: Função de saída

Outra função extremamente simples é aquela que determina que a opção é inválida, basta que ela exiba um aviso para o usuário e reexiba o menu. Para evitar que a mensagem exibida seja apagada muito rápido, eu decidi incluir uma parada no fim de cada comando, que é simplesmente um prompt que não faz nada, aguarda apenas que a tecla Enter seja pressionada para continuar a execução. Como esta parada vai ocorrer várias vezes, eu decidi criar uma função para ela também. As funções de opção inválida, de pausa de execução e de cadastro vazio estão na listagem 5.5. Em realidade a função de opção inválida é desnecessária, é uma única linha de código, mas isso facilita se quisermos alterar o comportamento do sistema no futuro.

```
1 function opcaoInvalida() {
2   console.log("Opção Inválida! Seleccione outra opção.")
3   pausa()
4 }
5
6 function cadastroVazio() {
7   console.log("Não há jogadores cadastrados.")
8   pausa()
9 }
10
11 function pausa() {
12   console.log()
13   prompt("Pressione ENTER para continuar...")
14 }
```

Listing 5.5: Funções para exibir mensagens ao usuário

Agora vamos começar a fazer o CRUD de fato. Primeiro, a função de adicionar um jogador ao cadastro está escrita na listagem 5.6.

```
1 function adicionarJogador() {
```

```
2  banner("ADICIONAR")
3
4  var nome  = prompt("Nome:  ")
5  var idade = prompt("Idade: ")
6  var sexo  = prompt("Sexo:  ")
7
8  cadastro.push({nome, idade, sexo})
9
10
11  console.log()
12  console.log(nome + " foi adicionado.")
13  pausa()
14 }
```

Listing 5.6: Função para adicionar jogador

Esta função exibe um letreiro com o nome da operação (o código da função `banner()` está na listagem 5.10, fica como desafio entender seu funcionamento) e em seguida obtém as informações do jogador guardando nas variáveis `nome`, `idade` e `sexo`.

O dicionário é formado por pares `"chave":valor` e é escrito entre chaves. O dicionário `const pessoa = {"nome":"Janio", "idade":41}` possui duas “posições” chamadas `nome` e `idade`, a primeira com valor `"Janio"` e a segunda com valor `41`. Para acessar os valores de um dicionário temos duas opções, escrever `dicionario.chave` ou `dicionario["chave"]`. No exemplo anterior, `pessoa.nome` retorna o valor `"Janio"` e `pessoa["nome"]` também retorna o mesmo valor.

Ao escrever `{nome, idade, sexo}`, na linha 8 da listagem 5.6, é criado um dicionário cujas chaves são os nomes das variáveis e cujos valores são os das variáveis correspondentes.

Na listagem 5.7 temos a função para listar os jogadores.

```
1  function listarJogadores() {
2    banner("LISTAR")
3
4    for (let i = 0; i < cadastro.length; i++) {
5      exibirJogador(cadastro[i], i+1)
6    }
7
8    pausa()
9  }
10
11 function exibirJogador(jogador, numero){
12   console.log("Número: " + numero)
13   console.log("Nome:    " + jogador.nome)
14   console.log("Idade:   " + jogador.idade)
15   console.log("Sexo:    " + jogador.sexo)
```



```
16 console.log("-----")
17
18 }
```

Listing 5.7: Função para listar jogadores

Esta função simplesmente itera por cada elemento do cadastro e exibe na tela usando a função `exibirJogador`. A função de exibição recebe o jogador e o seu número no cadastro.

Para apagar um jogador, o usuário deve selecionar qual jogador deseja apagar através de seu número no cadastro. Provavelmente é interessante exibir a lista de jogadores para facilitar a escolha. O código encontra-se na listagem 5.9.

```
1 function apagarJogador() {
2   banner("APAGAR")
3
4   var numero = prompt("Digite o número do usuário que deseja
      remover: ")
5   console.log()
6   console.log(cadastro[numero-1].nome + " foi removido.")
7
8   cadastro.splice(numero-1, 1)
9
10  pausa()
11 }
```

Listing 5.8: Função para remover jogador

O método `splice(idx, qtde)` remove da lista uma quantidade `qtde` de elementos a partir da posição `idx`. Seria interessante verificar se o número escolhido é válido, mas não vou fazer agora.

Por fim, a edição dos dados do jogador está implementada na listagem ???. E é a que dá um pouco mais de trabalho.

```
1 function editarJogador() {
2   banner("EDITAR")
3
4   var numero = prompt("Digite o número do usuário que deseja
      editar: ")
5   console.log()
6   console.log("Digite os novos valores. Deixe em branco para
      manter o valor atual.")
7
8   var jogador = cadastro[numero-1]
9   var nome = prompt("Nome (" + jogador.nome + "): ")
10  var idade = prompt("Idade (" + jogador.idade + "): ")
11  var sexo = prompt("Sexo (" + jogador.sexo + "): ")
12 }
```

```
13  if (nome=='') {nome = jogador.nome}
14  if (idade=='') {idade = jogador.idade}
15  if (sexo=='') {sexo = jogador.sexo}
16
17  cadastro[numero-1] = {nome, idade, sexo}
18
19  console.log()
20  console.log("Jogador " + numero + " foi editado.")
21  console.log()
22  exibirJogador(cadastro[numero-1], numero)
23  pausa()
24 }
```

Listing 5.9: Função para editar jogador

Assim como na remoção, o usuário deve escolher pelo número o jogador que deseja editar. Aqui também seria interessante ter uma lista dos jogadores para facilitar a escolha. Como na criação de um jogador novo, são solicitados os dados do jogador, com a diferença que os valores atuais são exibidos entre parênteses.

As linhas condicionais seguintes fazem com que o valor novo seja igual ao antigo caso o usuário tenha deixado a informação em branco. Por fim, os novos dados são inseridos na posição selecionada do registro.

A única função que falta definir é **banner**, que está na listagem 5.10. Ela é meramente estética e pode ser ignorada e todas as chamadas a ela removidas. Ela cria uma espécie de letreiro com o texto dado pelo primeiro argumento e a largura dada pelo segundo argumento.

```
1  function banner(txt, tam=15){
2    if (tam < txt.length + 2) {
3      tam = txt.length + 2
4    }
5
6    const rpt = (tam-txt.length)/2
7    const off = (tam-txt.length)%2
8
9    console.log("/") + "=".repeat(tam) + "\\")
10   console.log("|" + " ".repeat(rpt) + txt + " ".repeat(rpt +
11     off) + "|")
12   console.log("\\\" + "=".repeat(tam) + "/"")
13   console.log()
14 }
```

Listing 5.10: Função para exibir título

Bom, agora temos um CRUD funcionando e já podemos fazer as operações sobre nossos jogadores. Espero que você seja capaz de modificar este programa para acrescentar mais informações sobre o jogador, mudar o tipo de

objeto cadastrado, ou a forma de exibição. Brinque um pouco e veja o que é capaz de fazer. Como fazer se eu quiser acrescentar um email para cada jogador? E se eu quiser cadastrar itens do estoque de uma loja ao invés de jogadores? Como listar todos os jogadores que têm uma certa idade? Acredito que você consegue fazer tudo isso com um pouco de esforço.

5.2 Persistência de Dados

Tudo muito legal, cadastramos todos os nossos jogadores, fechamos o nosso programa e, quando abrimos ele novamente para utilizar, o cadastro está vazio! O que aconteceu?!

Bem, em qualquer aplicativo, escrito em qualquer linguagem de programação, todas as variáveis, constantes, valores, que são declarados no código deixam de existir no momento em que a aplicação é encerrada. A memória RAM, que é onde estão estas informações durante a execução do aplicativo, é volátil.

5.2.1 Arquivos

Para resolver este problema, precisamos guardar as informações que não desejamos perder em uma memória mais estável, de longa duração. O disco rígido (HD), ou um cartão de memória, são exemplos disso.

Nestas memórias as informações são guardadas em blocos chamados de arquivos e organizados em estruturas hierárquicas chamadas pastas, ou diretórios.

A função da estrutura de pastas, assim como os nomes e extensões dos arquivos são para ajudar na organização da informação.

De forma muito abrangente, os arquivos são a forma mais genérica de guardar informações de forma persistente. Não existe forma correta ou errada de armazenar informações em um arquivo, tudo depende do que você pretende fazer depois.

A organização e exibição de dados é chamada de Arquitetura da Informação. Parece uma coisa complicada mas não é, faz parte do nosso dia a dia. Um livro, por exemplo, é dividido em capítulos e os capítulos em seções e as seções em subseções, tudo isso visa facilitar a leitura e a organização das informações apresentadas.

Um dicionário, por exemplo, seria muito pouco útil se não fosse organizado de forma alfabética, mas ainda seria um dicionário. As palavras e frases, escrevemos da esquerda para a direita, porque assim foi convencionado na nossa língua. No entanto, nada impede que eu armazene uma frase

ao contrário ou que eu escreva primeiro as letras ímpares e depois as pares. Tudo é uma questão de escolha. O importante, no fim das contas, é que um padrão seja adotado e que ele seja adequado à finalidade que se deseja.

O processo de leitura de uma informação e transformação do seu conteúdo em objetos de uma linguagem de programação é chamado de *parsing*. Um *parser* é uma função que converte o conteúdo em objetos da linguagem. Normalmente guardamos os dados de tal modo que seja fácil fazer esse processo.

Quando um conjunto de dados tem uma estrutura padronizada de organização costumamos chamá-lo de banco de dados. De forma ampla, todo arquivo é um tipo de banco de dados.

5.2.2 Escrevendo Arquivos no JavaScript

No JavaScript, podemos trabalhar com arquivos usando o módulo `fs`. Neste módulo temos diversas métodos relativos a operações com arquivos e pastas, vou apresentar apenas os mais básicos deles.

Em todos os métodos listados a seguir, `nome` é uma string contendo o caminho e o nome do arquivo. Se apenas o nome for dado, é utilizada a pasta atual como caminho.

- `readFileSync(nome, <codificação>)`: retorna o conteúdo do arquivo, a codificação é opcional, mas para nossos exemplos vamos usar `'utf8'`, que é o padrão utilizado para escrita.
- `writeFileSync(nome, dados)`: escreve a string `dados` no arquivo. Se o arquivo já tinha algum conteúdo anteriormente, este conteúdo é apagado.
- `appendFileSync(nome, dados)`: escreve a string `dados` no fim do arquivo. Se o arquivo já tinha algum conteúdo anteriormente, este conteúdo é mantido.
- `unlink(nome)`: apaga o arquivo

A seguir temos alguns exemplos de como utilizar estes métodos.

```
1 const fs = require('fs') //importa o módulo fs
2
3 //abre o arquivo 'teste.txt' e escreve 'Olá!' nele
4 fs.writeFileSync('teste.txt', "Olá!")
5 //abre o arquivo 'teste.txt' e acrescenta uma linha
6 fs.appendFileSync('teste.txt', "\nMais uma linha.")
7
8 //lê o conteúdo do arquivo 'teste.txt'
```

```
9 var data = fs.readFileSync('teste.txt','utf8')
10 //exibe os dados lidos
11 console.log(data)
12
13 //abre o arquivo 'teste.txt' e escreve 'Oi!' nele
14 fs.writeFileSync('teste.txt', "Oi!")
15 //lê o conteúdo do arquivo 'teste.txt'
16 data = fs.readFileSync('teste.txt','utf8')
17 //exibe os dados lidos, percebe que o que tinha antes foi
    apagado
18 console.log(data)
19
20 //apaga o arquivo 'teste.txt'
21 fs.unlinkSync('teste.txt')
```

Listing 5.11: Funções com arquivos

A string `\n` indica uma quebra de linha, se não utilizar isto o conteúdo novo seria escrito imediatamente após o anterior, sem pular uma linha. O que não é um problema, mas não era o que eu queria.

Bom, com isso poderíamos resolver o nosso problema, bastaria fazer uma função para escrever as informações do cadastro em um arquivo e, toda vez que o programa iniciar, ler as informações deste arquivo. Vamos tentar fazer isso.

Eu vou organizar meus dados no arquivo da seguinte forma, as informações de um mesmo jogador serão separadas por vírgula e escritas na ordem nome, idade e sexo. Os jogadores serão separados por quebras de linha. Escolher uma organização é essencial para que a gente consiga recuperar as informações depois. A listagem 5.12 tem a função para salvar o cadastro.

```
1 function salvarCadastro(){
2   var data = ""
3   for (let i = 0; i < cadastro.length; i++) {
4     data = data + cadastro[i].nome + ',';
5     data = data + cadastro[i].idade + ',';
6     data = data + cadastro[i].sexo + '\n'
7   }
8   fs.writeFileSync('cadastro',data)
9 }
```

Listing 5.12: Função para salvar o cadastro em arquivo

Essa função itera sobre cada jogador do cadastro e acrescenta os valores à string `data`, colocando ponto e vírgula após os dois primeiros valores e uma quebra de linha após o último valor de cada jogador. Perceba que o arquivo terá uma linha em branco no fim, porque ele sempre termina com a quebra de linha. Agora vamos fazer a recuperação dos dados do arquivo na listagem 5.13.

```
1 function carregarCadastro() {  
2   try{  
3     const data = fs.readFileSync('cadastro','utf8')  
4  
5     const jogadores = data.split('\n')  
6  
7     for (let i = 0; i<jogadores.length-1; i++){  
8       const info = jogadores[i].split(';')  
9       const nome = info[0]  
10      const idade = info[1]  
11      const sexo = info[2]  
12      cadastro.push({nome, idade, sexo})  
13    }  
14  }  
15  catch(error) {  
16  }  
17 }
```

Listing 5.13: Função para recuperar o cadastro salvo em arquivo

O bloco de comandos está dentro de uma estrutura chamada `try {} catch {}`. Essa estrutura é utilizada para tratar erros, tenta-se executar o que está dentro do bloco `try` e caso um erro ocorra, faz-se o que está no bloco `catch`. No nosso caso, caso o arquivo de cadastro não exista, a leitura provocará um erro. Nesse caso não precisamos fazer nada, basta ficar com o cadastro vazio que já temos, por isso o bloco `catch` está vazio.

A função `split(separador)` divide uma string em uma lista de strings a cada vez que encontra o separador. Então o comando na linha 4 separa o conteúdo do arquivo em blocos a cada quebra de linha encontrada, ou seja, a cada jogador. Há uma string vazia no final dessa lista, porque como eu disse antes, o formato que eu usei para salvar termina com uma quebra de linha. Por isso, a iteração na linha 6 vai até o número de `jogadores` menos um.

Na linha 7 a string que representa as informações de um jogador é dividida em uma lista usando o ponto e vírgula como separador. Então agora `info` é uma lista com três elementos, correspondentes aos três valores salvos. O que é feito agora é o processo de inserção de cada um dos jogadores que estavam no arquivo.

Na minha implementação, eu coloquei o carregamento do cadastro antes do laço `while` externo e o salvamento quando sai do programa. No entanto, provavelmente seria mais interessante atualizar o arquivo ao executar cada uma das operações CRUD. Da forma preguiçosa que fiz, se o programa for interrompido por algum erro e não terminar pela opção “Sair”, as edições feitas serão perdidas.

5.2.3 JSON

Bom, bem simples nossa solução, mas ela precisaria ser reescrita para cada alteração do CRUD, se acrescentarmos mais informações ou mudarmos a estrutura utilizada para armazenar os dados.

Felizmente, o JavaScript possui uma notação padrão que, hoje em dia, é muito utilizada para a troca de informações. JSON é a sigla para JavaScript Object Notation, que nada mais é que uma forma padronizada para descrever objetos como estes que estamos utilizando em nosso CRUD.

Ela utiliza basicamente a mesma sintaxe do JavaScript que utilizamos para escrever Arrays e Dicionários. Utilizando colchetes para as listas e pares chave-valor para os dicionários, representados entre chaves.

Uma grande vantagem de utilizar esse formato, que também é baseado em texto, como o que fizemos anteriormente, é que a conversão dos objetos em texto e do texto em objetos é feita de forma eficiente por métodos da própria linguagem.

- `JSON.stringify(objeto)`: converte o objeto em uma string.
- `JSON.parse(string)`: converte uma string, escrita no formato adequado, em objeto.

Utilizando esta facilidade, nossas funções de salvar e carregar o cadastro seriam bem mais simples, como temos na listagem 5.14.

```
1 function salvarCadastro(){
2   const data = JSON.stringify(cadastro)
3   fs.writeFileSync('cadastro',data)
4 }
5
6 function carregarCadastro(){
7   try{
8     const data = fs.readFileSync('cadastro','utf8')
9     cadastro = JSON.parse(data)
10  }
11  catch(error){
12  }
13 }
```

Listing 5.14: Funções para salvar e carregar o cadastro em arquivo usando JSON

Depois das modificações que fizemos, a estrutura geral do nosso CRUD fica como consta na listagem 5.15. As funções chamadas foram definidas ao longo deste capítulo.

```
1 const prompt = require('prompt-sync')()
2 const fs = require('fs')
3
4 var cadastro = []
5 var continuar = true
6
7 carregarCadastro()
8
9 while (continuar) {
10     const opcao = menu()
11
12     switch(opcao) {
13         case '0':
14             salvarCadastro()
15             sair()
16             break
17
18         case '1':
19             adicionarJogador()
20             salvarCadastro()
21             break
22
23         case '2':
24             if (cadastro.length == 0){
25                 cadastroVazio()
26                 break
27             }
28             listarJogadores()
29             break
30
31         case '3':
32             if (cadastro.length == 0) {
33                 cadastroVazio()
34                 break
35             }
36             listarJogadores()
37             editarJogador()
38             salvarCadastro()
39             break
40
41         case '4':
42             if (cadastro.length == 0) {
43                 cadastroVazio()
44                 break
45             }
46             listarJogadores()
47             apagarJogador()
48             salvarCadastro()
49             break
```



```
50
51     default:
52         opcaoInvalida()
53         break
54     }
55 }
```

Listing 5.15: Estrutura atualizada do CRUD

Capítulo 6

Bancos de Dados

Como discutimos anteriormente, a maioria dos sistemas precisa de alguma forma armazenar dados. Existem diversas formas de fazer isso e, inicialmente, os sistemas de arquivos eram usados.

Apesar de ser bastante simples, como vimos no capítulo anterior, há diversos problemas com os sistemas de arquivos quando não são bem implementados.

Se necessitamos compartilhamento de dados entre vários usuários, pode haver inconsistências se duas ou mais alterações simultâneas ocorrem nos arquivos. O mesmo ocorre quando precisamos compartilhar dados entre vários aplicativos.

Nestes caso, poderíamos ter várias cópias dos arquivos mas, além de gastar mais memória, não teríamos como garantir que todas as cópias sejam sempre atualizadas para todos a cada alteração.

Outro ponto é que, do modo que fizemos, é necessário reescrever toda a base a cada alteração, o que é inviável para grandes volumes de dados.

A segurança dos dados também é outro ponto importante, com um sistema de arquivos simples os dados estão disponíveis para qualquer um que saiba onde procurar, a menos que algum sistema de criptografia seja implementado.

De modo geral, trabalhar com sistemas de arquivos tradicionais aumenta a complexidade na construção e manutenção das aplicações, pois é necessário implementar mecanismos para manter a consistência, evitar redundância e garantir a segurança dos dados.

Neste ponto surgem os Sistemas Gerenciadores de Bancos de Dados (SGBDs) que hoje são muitas vezes usados como sinônimo de Banco de Dados, apesar de que o termo Banco de Dados seja muito mais abrangente em sua origem.

Os SGBDs são uma coleção de programas para definir, construir e manipular bases de dados de forma eficiente e segura.

Essencialmente, ainda são um sistema de arquivos, mas toda a parte de gerência, segurança, consulta e manutenção está bem implementada e segue padrões internacionais, sendo independente da linguagem de programação utilizada para desenvolver o aplicativo.

6.1 Tipos

Temos basicamente dois tipos de SGBDs, os relacionais e os não-relacionais. SGBDs relacionais armazenam seus dados em tabelas e são baseados na teoria matemática das relações, vindo daí o seu nome. Sua definição é ideal para garantir a consistência e possuem uma linguagem própria de criação, manipulação e administração dos dados chamada de SQL (*Structured Query Language*). Este tipo de SGBD ainda é o mais utilizado nas aplicações computacionais. Os representantes mais conhecidos desta categoria são, provavelmente, Oracle, SQL Server, MySQL e PostgreSQL.

Os bancos de dados não-relacionais são soluções para situações nas quais os bancos relacionais não atendem. Um exemplo são os ambientes com dados mistos (imagens, mapas e tabelas), que não podem ser tabulados em linhas e colunas de forma eficiente. São bastante utilizados em soluções distribuídas, na nuvem e, em geral, possuem uma estrutura mais flexível que os bancos relacionais. Também são conhecidos como NoSQL (Not Only SQL) e buscam consistência nas informações armazenadas, disponibilidade do banco de dados e tolerância ao particionamento das informações. Os representantes mais conhecidos desta categoria são MongoDB, Redis e Cassandra.

Existem vários tipos de SGBDs não-relacionais, que diferem quanto à forma de armazenamento e conceitos de modelagem. Os quatro tipos básicos são: orientado a documentos, chave-valor, representado por grafos e modelo colunar.

O NoSQL orientado a documentos consiste numa estrutura baseada em uma coleção de documentos, descritos através do JSON, sendo um documento um objeto que contém um código único com um conjunto de informações, podendo ser strings, documentos aninhados ou ainda listas. Exemplos dessa classe são o MongoDB e o CouchBase.

No NoSQL chave-valor temos uma modelagem que indexa os dados a uma chave. Ao se armazenar os dados, sua forma de procura se dá de forma similar a um dicionário. Esta forma de armazenamento permite a inserção de dados em tempo de execução, sem conflitar o banco e não influenciando na disponibilidade, pois seus valores são isolados e independentes entre si. Alguns exemplos deste tipo são o Oracle NoSQL, Riak, Azure Table Storage, BerkeleyDB e Redis.

o NoSQL representado por grafos utiliza três componentes básicos para o armazenamento. Os nós representam os dados, as ligações representam as associações entre os elementos e os nós e arestas têm atributos. Muito utilizado onde há dados fortemente ligados. Costuma ser vantajoso para consultas complexas, pois seu diferencial é o ganho de performance. Exemplos deste tipo são o Neo4J, OrientedDB, GraphBase e InfiniteGraph.

Por fim, o NoSQL de modelo colunar basicamente é uma tabela que possui várias famílias de colunas e, dentro dessas famílias, colunas onde estão as propriedades. Neste modelo as entidades são representadas por tabelas e os dados são indexados por uma tripla (linha, coluna, *timestamp*), que permite verificar diferentes versões de um mesmo dado. Os valores das propriedades das colunas são semelhantes ao modelo chave-valor. Também são indicados para problemas que envolvem consultas complexas e são bastante utilizados em mídias sociais.

6.2 Propriedades ACID

Independentemente do tipo utilizado, o SGBD deve garantir confiabilidade nas transações, já que várias podem ocorrer simultaneamente.

Uma transação é uma sequência de operações executadas como uma única unidade lógica de trabalho, ou seja, são diversos procedimentos executados pelo SGBD mas que, para o usuário, é percebido como uma única ação.

Uma transação deve atender quatro propriedades: atomicidades, consistência, isolamento e durabilidade. Por causa das iniciais, estas propriedades das transações são conhecidas como ACID.

A atomicidade diz respeito ao fato de que todas as ações da transação devem ser executadas com sucesso para que ela seja efetivada na base e persista no banco (*commit*). Se pelo menos uma ação falha, a transação inteira é desfeita (*rollback*).

A consistência diz que cada transação deve gerar um novo estado válido, ou seja, que atende às regras do banco de dados, caso contrário ela não pode ser efetivada.

O isolamento garante que transações simultâneas não afetem umas às outras. Cada uma é independente das demais.

A durabilidade faz com que o sistema registre todos os dados validados, para que estejam disponíveis no estado correto em caso de falha ou reinício do sistema.

6.3 O Modelo Relacional

A modelagem de bancos de dados, ou seja, decidir como as informações serão representadas e armazenadas, é um problema similar à modelagem de classes. Mas aqui temos que levar em conta as restrições impostas pelo tipo de SGBD escolhido. No caso de SGBDs NoSQL como o MongoDB praticamente nenhum trabalho extra de modelagem é necessário, visto que elas armazenam dados no formato JSON de descrição de objetos.

No entanto, há problemas genéricos que independem do modelo escolhido. Como o modelo relacional é, ainda, o mais utilizado, vou considerar a modelagem de dados levando em conta as especificidades do modelo relacional.

6.3.1 Conceitos Fundamentais

Os dados são guardados em Tabelas, organizadas em linhas e colunas. Cada linha representa uma entidade de um conjunto, enquanto cada coluna representa um atributo das entidades. Portanto, uma célula é um atributo de uma entidade específica.

Um conjunto de uma ou mais colunas cujos valores são capazes de determinar unicamente uma entidade é chamado de Chave Primária. Em geral é recomendado que a chave primária seja um elemento independente, sem significado real para o problema.

Para fazer referência a uma outra Tabela, podemos inserir a chave primária de uma tabela como um atributo da outra. Com essa referência, podemos buscar informações na tabela referida para complementar as informações da entidade. Neste caso, a referência é chamada de Chave Estrangeira.

Por exemplo, considere uma tabela de departamentos que possui os atributos código, nome e chefe. Provavelmente código é uma chave primária desta tabela. Agora considere uma tabela de funcionários que tem os atributos matrícula, nome do funcionário e código do departamento onde está lotado. O código do departamento é uma chave estrangeira, pois podemos usar este código para descobrir, na tabela de departamentos, o nome do departamento e a chefia.

Através das chaves estrangeiras podemos fazer relações de um para muitos. No exemplo anterior, um departamento tem muitos funcionários, mas um funcionário tem um único departamento.

Agora imagine as entidades pessoa e assunto, cada uma com diversos atributos, e o relacionamento “interesse” entre elas. Muitas pessoas podem se interessar por um mesmo assunto e uma pessoa pode se interessar por vários assuntos. Neste caso não tem como utilizar a chave estrangeira para representar o relacionamento.

A solução ingênua seria representar tudo numa tabela só, mas nesse caso precisaríamos replicar diversos dados. Por exemplo, uma linha diria que Jânio Coutinho Canuto, de 41 anos, professor, brasileiro, tem interesse em música. A outra diria que Jânio Coutinho Canuto, de 41 anos, professor, brasileiro, tem interesse em computação. Isso seria muito redundante e passível de erros, pois os dados de uma mesma pessoa poderiam diferir de uma linha para a outra.

Neste caso o que fazemos é utilizar uma entidade associativa ou Tabela Associativa. Esta tabela apresenta, como chaves estrangeiras, as chaves primárias das duas tabelas que estamos relacionando. E sua chave primária é, em geral, o conjunto das chaves estrangeiras.

Se Jânio tem código 01, música tem código MUS e e computação tem código COMP, a tabela associativa deve ter uma linha com valores 01 e MUS e outra com valores 01 e COMP.

6.3.2 Propriedades

No modelo relacional algumas propriedades devem ser sempre verdade:

- Cada tabela em um banco de dados possui um nome único.
- Cada coluna em uma tabela possui um nome único.
- Os valores de uma coluna pertencem a um mesmo domínio.
- A ordem das colunas e linhas é irrelevante.
- Não há duas linhas iguais.
- Nenhuma chave primária pode ser vazia.
- Cada célula pode ser vazia (se não for uma chave primária) ou conter um único valor do domínio.
- A chave estrangeira ou tem o valor de uma chave primária existente ou é vazia.

6.3.3 Modelagem de Entidades e Relacionamentos

Como já vimos, o processo de modelagem de dados é similar ao da modelagem de classes. As tabelas representam um conjunto de entidades que possuem atributos em comum e os relacionamentos são os tipos de associações entre eles.



Figura 6.1: Relacionamento de lotação entre professores e departamentos

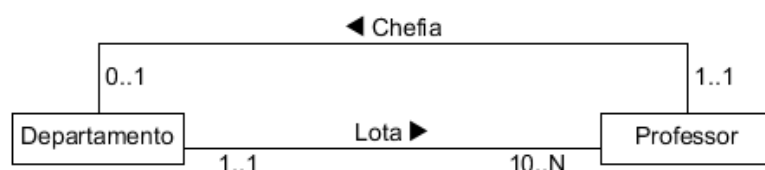


Figura 6.2: Dois relacionamentos entre duas entidades

Relacionamentos

Os relacionamentos possuem uma propriedade chamada de “cardinalidade” que indica o número mínimo e máximo de associações possíveis. Por exemplo, na UFS, um professor está lotado em um único departamento, enquanto um departamento lota no mínimo 10 e no máximo uma quantidade arbitrária, que chamaremos de N, de professores.

A cardinalidade mínima aponta a quantidade de instâncias mínimas para que a associação seja estabelecida. No exemplo anterior, quando um professor é registrado no sistema ele tem que estar obrigatoriamente em um departamento. Quando um departamento é registrado, pelo menos 10 professores têm que ser lotados nele. Esse relacionamento está representado graficamente na figura 6.1.

Entre duas entidades pode haver mais de um relacionamento distinto. No caso do exemplo anterior, poderíamos ter um relacionamento de chefia entre professores e departamentos. Um professor pode chefiar ou não seu departamento, enquanto um departamento é chefiado por um professor. Esses relacionamentos estão representados na figura 6.2.

Além disso, uma entidade pode se relacionar com qualquer outra, inclusive consigo mesma (auto-relacionamento). Por exemplo, uma disciplina pode ser pré-requisito para várias outras disciplinas, e uma disciplina pode ter várias disciplinas como pré-requisito. Isso está ilustrado na figura 6.3.

Como dissemos antes, alguns relacionamentos são representados por entidades associativas. Além disso, um relacionamento pode envolver mais de

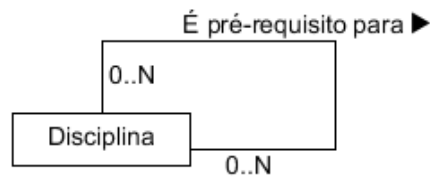


Figura 6.3: Auto-relacionamento

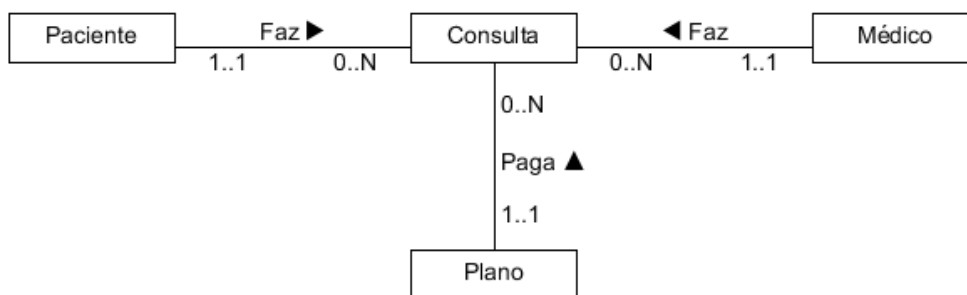


Figura 6.4: Entidade associativa

duas entidades. Considere o caso em que um paciente faz consulta com vários médicos. Um médico faz consultas para vários pacientes. As consultas podem ou não ser pagas por um plano de saúde. O relacionamento “consulta” é mais facilmente representado através de uma entidade e envolve três entidades: médicos, pacientes e planos de saúde. Esse relacionamento está ilustrado na figura 6.4.

Atributos

Os atributos são as propriedades relevantes, para o problema, de uma entidade ou relacionamento. Em certas situações reais os atributos podem ser multi-valorados. Por exemplo, uma pessoa pode possuir vários números de telefone. No entanto, as propriedades do modelo relacional nos dizem que só pode haver um valor por célula. Podemos resolver este problema guardando os diversos valores como um longo texto, mas isso pode dificultar na hora de consultar os valores.

Uma solução mais interessante em situações como esta é a criação de uma tabela auxiliar para armazenar os atributos multi-valorados. No exemplo anterior, poderíamos ter uma tabela de telefones que utiliza o CPF da pessoa,



Figura 6.5: Relação de fornecimento

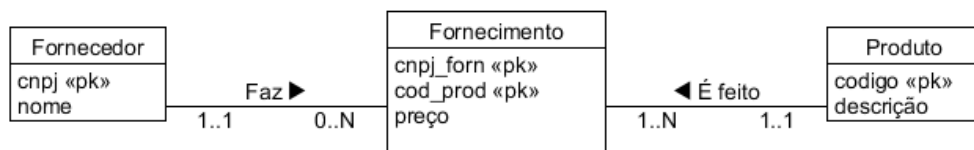


Figura 6.6: Relacionamento com atributo

ou qualquer que seja a chave primária da tabela de pessoas, como chave estrangeira.

Em certas situações é possível que um atributo seja vazio (valor *null*). Isto se dá quando a entidade de fato não possui aquele atributo ou o valor ainda não é conhecido.

Outro ponto importante é que relacionamentos podem possuir atributos, que são dados que não pertencem a nenhuma das entidades envolvidas no relacionamento. Nestes casos precisamos representar o relacionamento como uma tabela associativa.

Considere a seguinte situação: um fornecedor fornece um ou mais produtos, enquanto um produto é fornecido por zero ou mais fornecedores. Esta relação está representada na figura 6.5, nela o nome no topo é o nome da tabela e os nomes abaixo da linha são os atributos. O atributo seguido de «pk» é a chave primária da tabela.

Agora pense, o preço do produto é uma característica do produto ou do fornecedor?

Fixando um produto e variando os fornecedores o preço pode mudar, então não é uma característica do produto. Fixando o fornecedor e variando os produtos o preço certamente vai mudar, então não é um atributo do fornecedor. Se não é de nenhum dos dois, é uma característica do relacionamento, ou do par (fornecedor, produto). Isso está ilustrado na figura 6.6.

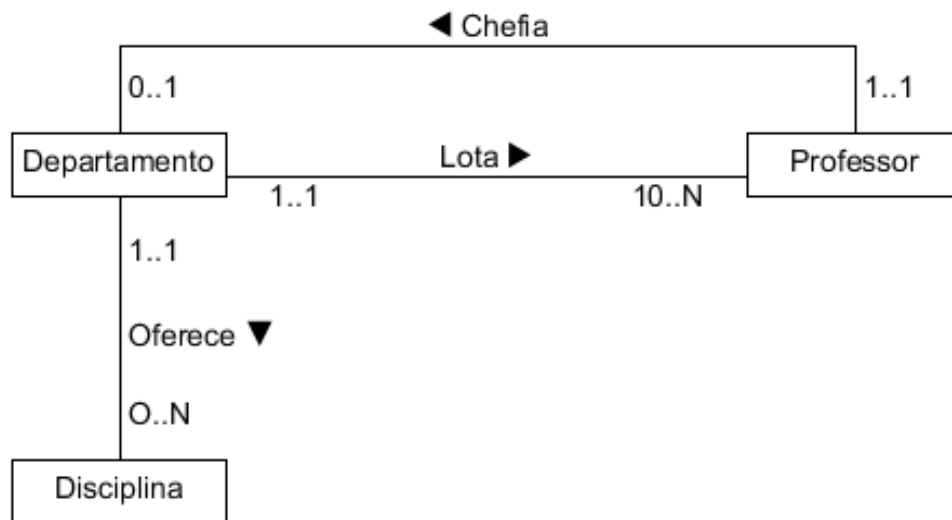


Figura 6.7: Um pequeno Banco de Dados acadêmico

Entidades

Outro ponto importante a ser considerado na modelagem é o seguinte: a informação que eu desejo representar deve ser um atributo de uma entidade ou uma entidade nova relacionada à primeira? Por exemplo, o departamento que oferece uma disciplina deve ser um atributo da entidade disciplina ou deve ser uma nova entidade?

Há alguns critérios que podem nos ajudar a decidir isso:

- O novo elemento tem atributos próprios?
- A entidade nova é relevante para o sistema?
- A nova entidade pode ser relacionada a diversas ocorrências da primeira?
- A nova entidade se relaciona com outras entidades do sistema, além da primeira?

Em geral, se algumas dessas perguntas é afirmativa é interessante modelar a nova informação como uma entidade à parte.

No exemplo em questão, todos os critérios são satisfeitos, então o departamento deve ser uma nova entidades, como mostrado na figura 6.7.

Restrições

Os diagramas de entidades e relacionamentos nem sempre são capazes de modelar toda informação que precisamos, em especial as restrições de integridade. Há dois tipos básicos de restrições de integridade, sobre relacionamentos e sobre valores.

A cardinalidade é uma das poucas restrições de integridade expressas no diagrama, as demais precisam ser escritas em linguagem natural no dicionário de dados do projeto.

Por exemplo, considere um modelo onde um aluno pode se matricular em 0 ou mais turmas mas, ele não pode ser matricular mais de uma vez na mesma turma e ele só pode se matricular se já cursou todos os pré-requisitos da disciplina daquela turma.

Tudo que vem depois do “mas”, no parágrafo anterior, são restrições de relacionamento que não podem ser descritas no diagrama. A primeira é uma restrição de repetição, que indica quantas vezes os mesmos dois elementos das entidades podem ser relacionados. A segunda é uma restrição de dependência, que indica que um relacionamento pode ser restringido por outro, ou depender de relacionamentos passados.

Além das relações, devemos indicar as restrições sobre os valores dos atributos. O domínio de um atributo muitas vezes pode ser representado no diagrama, sendo descrito ao lado do nome do atributo.

A descrição da restrição pode ser feita de diversas formas como enumeração, intervalos, ou regras para identificar se o valor é válido ou não. Em resumo, qualquer forma de definir um conjunto.

Nas linguagens dos SGBDs estas restrições costumam ser efetivadas através dos tipos de dados, mas também podem ser descritas no dicionário de dados do projeto. Além do próprio SGBD, que verifica a pertinência do dado fornecido ao domínio especificado, é interessante que o aplicativo que usa o SGBD faça uma validação dos dados entrados pelos usuários.

Além do domínio, pode ser interessante para a modelagem identificar os valores prováveis, ou improváveis. Isto pode ajudar a detectar anomalias nos dados. Por exemplo, se dizemos que a idade é um número inteiro e positivo, para o sistema é qualquer valor de 1 a infinito mas, para qualquer pessoa é improvável que esse valor seja muito superior a 100. Já num sistema de cadastro de professores numa universidade federal, é improvável que esse valor seja superior a 70, que é a idade da aposentadoria compulsória.

Outros aspectos importantes acerca dos atributos são a obrigatoriedade e a dependência. A primeira determina se um valor pode ser nulo ou não. Como já vimos antes nas propriedades dos SGBDs relacionais, chaves primárias nunca podem ser nulas. A segunda identifica casos em que o valor de um atri-

buto depende do valor de outro atributo. Por exemplo, a data de demissão de um funcionário não pode ser anterior à data de admissão do mesmo.

6.4 Conclusão

Com um pouco de bom senso, paciência e usando os conhecimentos que obtivemos até agora, devemos ser capazes de organizar os dados de forma razoável para nossa aplicação.

Lembre-se, se as regras estão sendo cumpridas não existe forma certa ou errada de fazer o modelo. Às vezes uma forma é melhor que outra a depender da aplicação que se está construindo. Outro ponto importante para se ter em mente é que não é raro que o modelo inicial precise ser alterado conforme o desenvolvimento avança. Situações e relacionamentos imprevistos podem surgir ao longo do processo.

Para aplicações independentes e pequenas, um sistema de arquivos com um bom controle de entrada pode ser melhor que usar um SGBD. Por exemplo, para guardar arquivos de configuração ou pequenas listas de dados.

No Apêndice A temos uma lista das principais funções da linguagem SQL para SGBDs relacionais.

Apêndice A

SQL

A linguagem SQL, cuja sigla significa *Structured Query Language* ou Linguagem de Consulta Estruturada possui comandos para criação, gerência e manipulação de bancos de dados.

Em geral as palavras-chave da linguagem são fáceis de entender, em inglês. Apesar de possuir uma padronização ISO, há pequenas diferenças de sintaxe a depender do SGBD específico que estamos utilizando. Neste documento eu adotei a sintaxe do MySQL.

Ainda sobre a sintaxe, algumas observações:

- todo comando SQL deve terminar com ponto-e-vírgula
- comentários podem ser inseridos com –
- as palavras chave não são sensíveis à caixa alta ou baixa (**SELECT** é igual a **select**), no entanto eu vou sempre escrever em caixa alta para facilitar a identificação das mesmas.

A seguir apresento os comandos mais utilizados da linguagem, sempre acompanhados da sua sintaxe e de algum exemplo de uso. Você pode testar os exemplos em qualquer compilador online de SQL ou no seu próprio sistema, caso tenha instalado. A Programiz tem um interpretador com uma interface legal.

A linguagem SQL tem comandos para administração de bancos, manipulação de tabelas e manipulação de dados. Primeiro são apresentadas as de manipulação de tabelas, para criação do esquema básico onde definimos os elementos, as relações, os atributos e suas restrições. Em seguida temos as de manipulação de dados, que é o que você vai precisar quando já tem um banco de dados pronto.

As funções de administração do banco serão ignoradas neste tutorial.

